

BACHELOR COMPUTER SCIENCE
SOFTWARE PROJECT

Research AI

Final Report

Triple One Intelligence

Jelmer André de la Porte	6578772
Ivan Balkenende	1196340
Jonathan Bosker	2046636
Anouar Boubkari	1747479
Lukas van Dee	1048708
Thijmen Ligter	4687582
Sybren van der Meijden	2077523
Onno Meppelink	0989711
Timon Zoutendijk	4611659

Client: Research Data Management (RDM) Support,
Information and Technology Services, Utrecht University

Supervisor: Dr. I.R. (Ioana) Karnstedt-Hulpus

June 22, 2026

Chapter 1**Abstract*****Lukas*****Abstract**

This report presents Research AI, a web application developed for Utrecht University that combines institutional research data with large language model (LLM) pipelines. Users can search for a researcher or organisation and generate textual outputs such as narrative profiles, NWO-style CVs, collaboration analyses and publication overviews. Next to the generated text, a connections panel shows the underlying institutional data directly, so the dataset itself remains visible at all times. Responses are streamed in real time and contain numbered citations that refer to publication records, while recognised names are rendered as internal links to other profile pages.

The frontend is a bilingual (English and Dutch) single-page application built with React and TypeScript, styled according to the Utrecht University styleguide and responsive on both desktop and mobile viewports. The backend exposes a REST API that performs retrieval-augmented generation over the institutional dataset.

Quality was verified through a layered test strategy with 903 tests across the unit, integration and system levels, reaching 96% backend code coverage. This was supported by mandatory peer-reviewed pull requests, three quality inspections and multiple rounds of user testing with domain experts provided by the client. The platform makes a clear distinction between AI-generated text and the underlying source data, which users can verify directly through the connections panel and cited publications.

Table of Contents

1 Abstract	2
2 Introduction	5
2.1. Software Project	5
2.2. Client	5
2.3. Objectives	5
2.4. Problem Statement	6
2.5. Solution Overview	6
2.6. Report Overview	6
3 Requirements	7
3.1. Functional Requirements	7
3.2. Non-functional Requirements	8
4 Architecture	10
4.1. Overview	10
4.2. Backend Architecture	10
4.3. Frontend Architecture	10
4.4. Deployment Architecture	11
5 Implementation	14
5.1. Technology Choices	14
5.2. Endpoints	15
5.2.1. Search and autocomplete	15
5.2.2. Entity connections	16
5.2.3. Internal vs. external entity split	16
5.2.4. Pagination	17
5.2.5. Streaming response	17
5.2.6. Abuse protection	18
5.3. Database	18
5.3.1. Node architecture	18
5.3.2. Schema of possible connections	19
5.3.3. Data harvesting	20
5.4. Pipelines	21
5.4.1. Object-oriented approach	22
5.4.2. Shared infrastructure across all pipelines	24
5.4.3. Helper functions	24
5.5. Large Language Model	28
5.6. File structure	28
5.7. Configuration and Environment file	29
5.8. Frontend Implementation	30
5.8.1. Technology Stack	30
5.8.2. Component Structure	31
5.8.3. File Structure	32
5.9. Deployment	32
5.9.1. Deployment Workflow	32
5.9.2. Container Configuration	33
5.9.3. Development environment	33
5.9.4. Production Environment	34
6 User interface	36
6.1. Visual design	36
6.2. Searching for entities	36

6.3.	Profile page	37
6.4.	Connections panel	38
6.5.	AI prompt categories	38
6.5.1.	Profiles and summaries	39
6.5.2.	Collaborations	39
6.5.3.	Publications	39
6.6.	Optional topic input	40
6.7.	Generated output, sources and navigation	40
6.8.	User documentation	41
6.9.	Language support	41
7	Test approach	42
7.1.	Quality aspects	42
7.2.	Functional testing	42
7.3.	Non-functional testing	43
7.4.	Functional test results	43
7.5.	Non-functional test results	44
7.6.	Retained elements	44
7.7.	Revised elements	44
8	Product Handover and Potential Future Development	45
8.1.	Handover Preparation	45
8.2.	Limitations	45
8.2.1.	Security	46
8.2.2.	Open Calls	46
8.3.	Future Development	46
9	Process	48
9.1.	Scrum Framework	48
9.2.	Sprint Backlog and Task Management	48
9.3.	Sprint Planning	49
9.4.	Daily Standups and Coordination	49
9.5.	Sprint Reviews	50
9.6.	Code Review and Integration	51
9.7.	Process of building Pipeline code	52
9.7.1.	Challenges	52
9.7.2.	Earlier approaches	52
10	Attachments	53
10.1.	Sprint summaries	53
10.1.1.	Sprint 1 Summary	53
10.1.2.	Sprint 2 Summary	58
10.1.3.	Sprint 3 Summary	63
10.1.4.	Sprint 4 Summary	68
10.1.5.	Sprint 5 Summary	73
10.1.6.	Sprint 6 Summary	78
10.1.7.	Sprint 7 Summary	83
10.1.8.	Sprint 8 Summary	88
10.1.9.	Sprint 9 Summary	94
10.2.	Project Plan	99
10.3.	Architecture Plan	120
10.4.	Test Plan	130
10.5.	Hour logging	140

Chapter 2

Introduction

Author: Sybren | Reviewer: Lukas

This section begins by introducing the context surrounding this project, after which we discuss existing problems within our domain to define our objectives. Having defined our goals, we can provide the problem statement with proposed solutions and end with an overview of this report.

2.1. Software Project

This software project was undertaken as part of Utrecht University's Computer Science Bachelor's programme. The group responsible for this project consisted of nine students working over a period of 20 weeks to gain experience and expand their knowledge.

2.2. Client

The project was developed for the Research and Data Management Services (RDMS) department, which is a branch of the Information and Technology Services (ITS) at Utrecht University. The RDMS department manages and develops research-supporting applications and infrastructure used within Utrecht University and, in some cases, by other institutions. Examples include applications such as Yoda¹ and Ricgraph². Given this institutional context, the system to be delivered will operate in an environment where various non-functional requirements are particularly important; reliability, portability, and compliance with institutional policies are key requirements.

2.3. Objectives

Universities produce and collect large amounts of research-related data, including academic publications, researcher employment histories, funding opportunities, and organizational affiliations. For organizations such as Utrecht University, acquiring this data is relatively straightforward. The real challenge lies in centralizing and structuring it, as data from multiple sources must be cleanly combined while dealing with the many errors and inconsistencies within those datasets.

Tools such as Ricgraph address part of this challenge by integrating databases like Pure³ and OpenAlex⁴ into a single knowledge graph database. Although this greatly simplifies working with research data, the resulting graph database still requires further enrichment depending on the intended use case. Ricgraph therefore provides a strong foundation, but not a complete solution.

The final and most complex challenge is making structured data practically accessible. Manually exploring multiple large databases whenever a subset of information is needed is not feasible. Solving this would enable a wide range of use cases: research managers could find suitable researchers for their projects, researchers could identify collaborators in their field, and policy advisors could explore connections through co-authored publications.

With the recent advancements in Artificial Intelligence and more specifically Large Language Models (LLMs) in combination with techniques like Retrieval Augmented Generation (RAG), an opportunity has presented itself to solve the problem of interactively presenting large amounts of data

¹Yoda is a research data management service developed by Utrecht University: <https://www.uu.nl/en/research/yoda>

²Ricgraph is a service which allows exploration of research data using a knowledge graph: <https://www.ricgraph.eu/what-is-ricgraph.html>

³Pure is Utrecht University's research information system: <https://www.uu.nl/en/university-library/advice-support-to/researchers/publishing-support/pure-research-information>

⁴OpenAlex is a free database containing large amounts of research data: <https://openalex.org/>

in an understandable way. LLMs can process data and output natural language responses, but this comes with difficulties. LLM outputs are non-deterministic, meaning that the same query can yield different responses each time. Additionally, LLMs are prone to generating plausible but factually incorrect information, commonly referred to as *hallucinations*. The non-deterministic nature of LLMs also makes testing, and therefore improving, output quality difficult, as results cannot be reliably reproduced. It is our objective to overcome these obstacles and solve the stated problem.

2.4. Problem Statement

Concretely, the problem is as follows: currently, no system exists that utilizes and processes all relevant research data to make common tasks, such as drafting a researcher profile, producing an overview of departmental collaborations, or preparing policy-oriented summaries, less complex and time-consuming. Tasks like these still require users to manually interact with large databases and compile their results themselves.

2.5. Solution Overview

Our project solves this problem by making a web application that allows users to interact with an LLM about research data. Users are provided with options to search for both researchers and research institutions, after which they can utilize predefined prompts to query the LLM about the selected entity. These responses are based on data retrieved from a custom database, with sources cited to ensure traceability and truthfulness. The database uses Ricgraph accompanied by further enrichment scripts to provide the LLM with as much useful context as possible.

2.6. Report Overview

The following sections explain and justify the decisions we have made, explain our resulting software in greater detail, and reflect upon the process we used to complete this project. These details are provided to thoroughly document this project for any future expansions and to make this project reproducible.

Chapter 3

Requirements

Author: Sybren | Reviewer: Lukas

This section outlines the requirements of the project to clarify its goals and scope. The requirements were derived by analyzing the objectives of this project and the needs of relevant stakeholders. A prototype of the system (see Figure 3.1) was also provided to further clarify the expected behaviour and functionality of the system. Rather than continuing development on the prototype, the team decided to create the system from the ground up. This decision was primarily motivated by the fact that the prototype had not been designed with all non-functional requirements in mind.

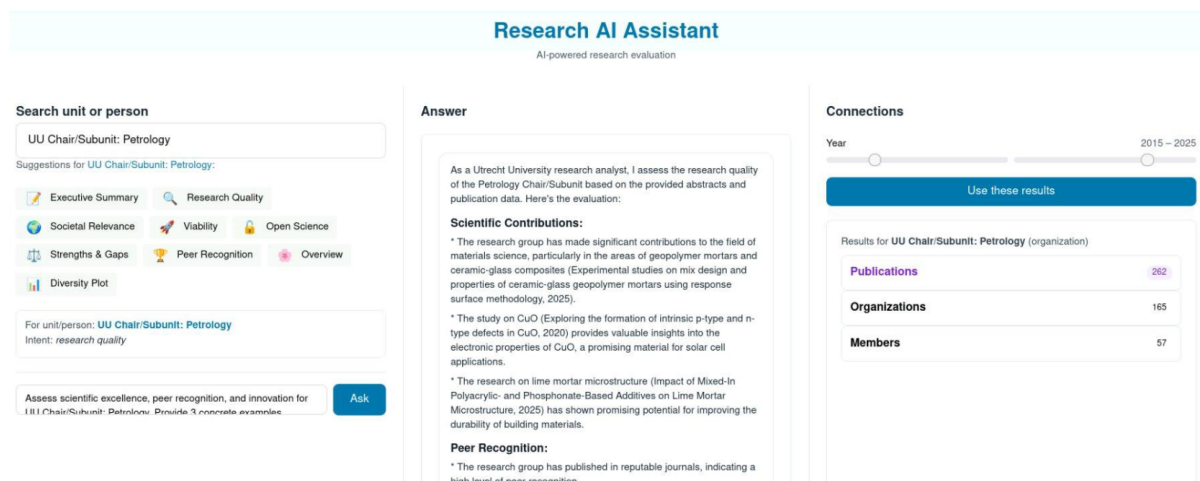


Figure 3.1: The client-provided prototype.

First, the functional requirements are presented using a MoSCoW prioritization. Afterwards, the non-functional requirements are described based on the needs of the project's various stakeholders.

3.1. Functional Requirements

The functional requirements in order of priority are as follows:

Must Haves

- A basic and usable UI design
- Integration with an LLM
- Integration with Ricgraph
- Integration with Pure/OpenAlex
- A set of pre-defined prompts for common research and policy use cases
- Find the top 20 organizations a faculty collaborates with
- Find publications, researchers, and organizations relevant to a researcher's own work (through abstract similarity)
- Generate a CV for a UU profile based on the researcher's publications
- Use a graph database
- RAG retrieval pipeline
- Search for persons and organizations

Should Haves

- Mobile-responsive UI
- Filtering of results (e.g., by year)
- Display individual researcher collaborations
- Show lists of collaborating researchers directly, without relying on the LLM
- Produce a list of research outputs with overlapping topics
- LLM Accountability section (explaining how the system arrived at its answer)
- Handling uncertainty ("no answer possible" from AI)
- Consistent AI outputs (similar prompts should produce reasonably similar outputs)

Could Haves

- UI redesign (instead of prototype UI)
- Clickable right pane (which enables easier relational navigation)
- User-facing AI model selection
- Improve the filtering algorithm for what data gets sent to the LLM
- Allow users to write their own prompts
- Import media, prizes, and activities data into the database
- Improve the search function for persons and organizations
- User-dependent prompts (one-click prompt submission based on the user's background)
- Refined set of standard prompts
- Visualization of collaboration networks
- Visualization of other data
- Support for showing open calls

Won't Haves

- Authentication via SURFconext¹ (to protect sensitive data)
- Extended conversations with AI
- Admin panel for user statistics

3.2. Non-functional Requirements

The RDMS department needs to be able to maintain and expand the product, and they also want the system to be usable by other universities. Furthermore, the client specified the technological tools that should be used. Considering that Utrecht University has many Dutch- and English-speaking students, the system should be bilingual and have the option to include more languages in the future. Both researchers and research staff value transparency and policy-aware AI behaviour, while also requiring reasonable performance. The needs of these stakeholders translate to the following non-functional requirements:

- Python backend²
- Typescript³ combined with React frontend⁴
- Open source

¹<https://www.surf.nl/en/services/identity-access-management/surfconext>

²<https://www.python.org/>

³<https://www.typescriptlang.org/>

⁴<https://react.dev/>

- Portable (e.g. it needs to support interchangeable LLMs)
- Maintainable (clearly structured and high quality code)
- Reliable (LLM outputs are accurate)
- Transparent (LLM output should clearly indicate what information was used)
- Bilingual design (with more possible languages in the future)
- Good performance (e.g. waiting no more than 20 seconds for LLM responses)

Chapter 4

Architecture

This chapter describes the general structure of the final product,

4.1. Overview

Onno

4.2. Backend Architecture

Onno

4.3. Frontend Architecture

Thijmen

The Research AI frontend is a React-based¹ single-page application, in alignment with the client's technological constraints. Three core principles drive the architecture: a clear separation of concerns through component composition, a unidirectional data flow from the API layer through custom hooks into UI components, and a responsive layout that adapts to both desktop and mobile viewports.

Following the separation-of-concerns principle, the interface is organized as a hierarchical component architecture (see Figure 4.1), structured across four distinct layers:

- **Page-level components:** The top-level containers mapped directly to specific application routes, managing high-level state and orchestrating multiple features into a cohesive page experience.
- **Feature-specific composite components:** Complex, assembled components that encapsulate specific business logic or major application functionalities by combining smaller primitive elements.
- **Primitive components:** The smallest, highly reusable, and stateless building blocks of the user interface, free of domain-specific logic to ensure visual and functional consistency across the application.
- **Layout components:** Components that define the structural shell of the application, establishing a consistent layout, navigation, and user experience framework across all routed pages.

The unidirectional data flow runs from the API layer, through custom hooks that encapsulate global stateful logic and side effects, into the UI components that consume it.

¹<https://react.dev/>

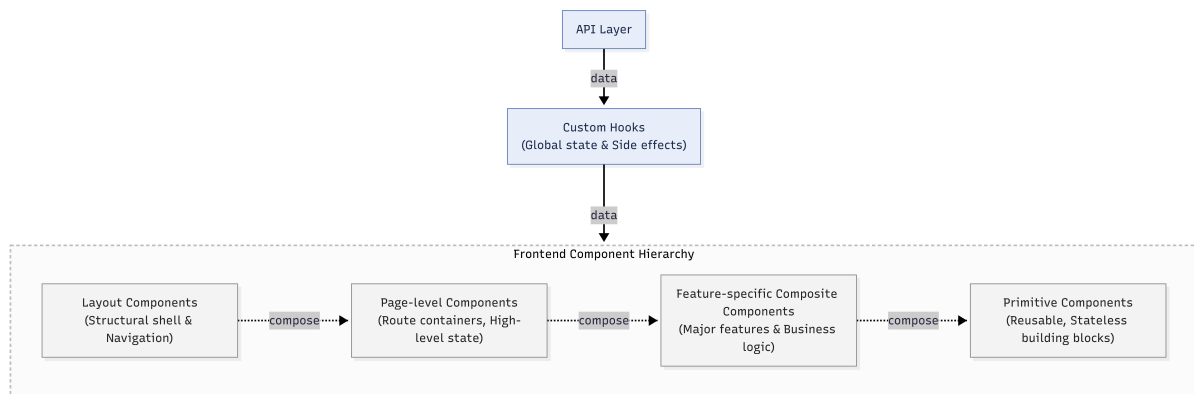


Figure 4.1: Visualization of the frontend architecture: four component layers assembled through composition, fed by a unidirectional data flow from the API layer through custom hooks into the UI.

At the application level, the architecture addresses several cross-cutting concerns:

- **Routing:** The application is structured around three primary views reached through client-side routing: a home and search view, an entity workspace for a specific person or organization, and an end-user documentation view.
- **Internationalization:** The interface is bilingual, supporting English and Dutch across all user-facing text.
- **Networking and streaming:** For the real-time streaming of model output, the architecture employs Server-Sent Events rather than WebSockets. Because the streaming requirement is strictly one-way, this choice removes the need to manage persistent connections and thereby simplifies deployment.

4.4. Deployment Architecture

Author: Jelmer | Reviewer: Timon

In the production setting, the application consists of four containers on a single Linux server (Figure 4.2).

- **Web frontend container.** A Caddy² container that serves the compiled React frontend as static files and acts as a reverse proxy, forwarding `/api/*` requests to the backend API.
- **Backend API.** A REST API built with Python 3.12 and FastAPI. It handles search queries, auto-complete and AI pipelines. AI-generated responses are streamed to the browser using Server-Sent Events (SSE).
- **Neo4j database.** A Neo4j³ container that stores graph data including researchers, publications, organizations and their relationships. Publication nodes, identified by their Digital Object Identifier (DOI), are extended with vector embeddings and abstract text, allowing for semantic similarity search queries (see Figure ??).
- **AI service.** An Ollama⁴ container running two models, namely `command-r:7b` for chat generation (on GPU) and `snowflake-arctic-embed2` for embedding generation (on CPU).

Table 4.1 lists the runtime connections between components.

²<https://caddyserver.com/>

³<https://neo4j.com/>

⁴<https://ollama.com/>

Production Architecture

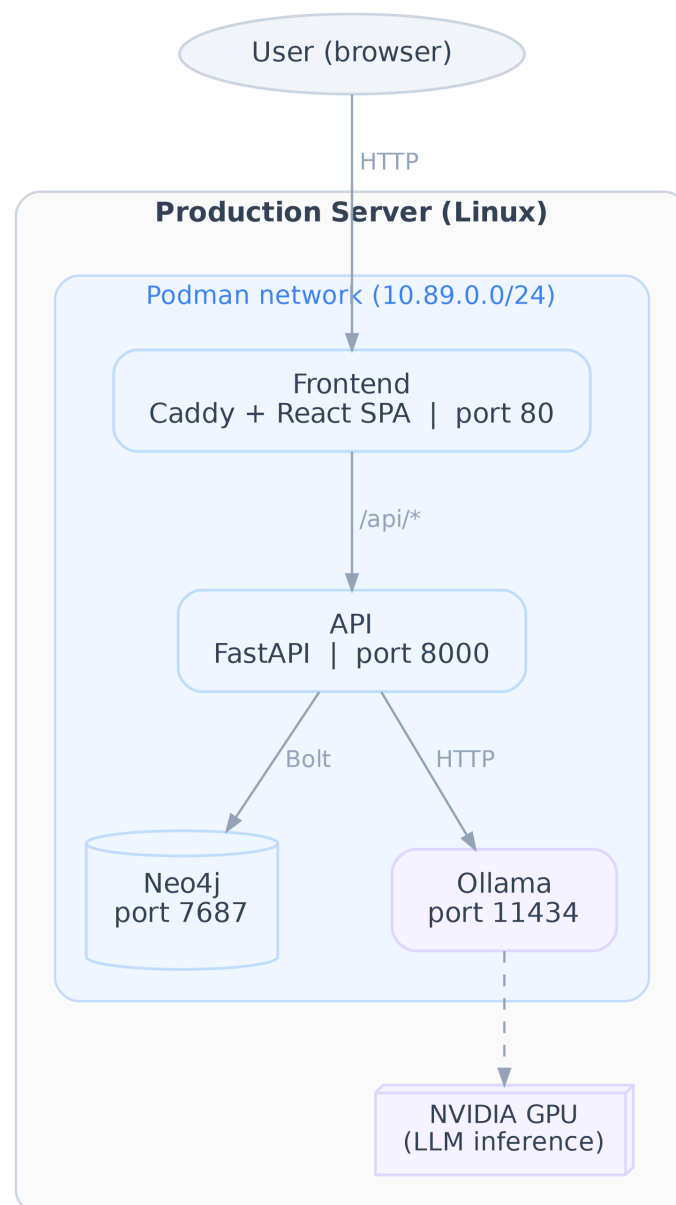


Figure 4.2: Production architecture. All four containers run on a single Linux server within a shared Podman network. Only the frontend container accepts incoming connections from the browser. All other services are internal to the Podman network.

From	To	Protocol	Purpose
Browser	Frontend	HTTP	User accesses the web application
Frontend	API	HTTP (reverse proxy)	Caddy forwards /api/* to port 8000
API	Neo4j	Bolt (7687)	Graph queries and vector similarity search
API	Ollama	HTTP (11434)	Embedding and chat generation

Table 4.1: Runtime communication between components.

Chapter 5

Implementation

5.1. Technology Choices

Author: Jelmer | Reviewer: Timon

This section describes the technologies used in the project and the reasoning behind each choice.

- **Neo4j and Ricgraph.** The use of Neo4j was determined by the client's requirement to integrate Ricgraph, which depends on Neo4j as its underlying database. Neo4j supports storing and searching vector embeddings natively, which allows the 1024-dimensional abstract embeddings to be stored directly on publication nodes. A single Cypher query can then traverse the graph and perform a semantic similarity search, without requiring a separate vector database. Both Ricgraph and the application run on the same Neo4j instance.
- **Python and FastAPI.** Python was selected as the backend language because the team had the most experience with it and because the client-provided prototype was written in Python. The Ricgraph library, which the project was required to use for data acquisition, is also a Python package. FastAPI was chosen over Flask for its support for asynchronous request handling, which is necessary for streaming AI responses to the browser via Server-Sent Events. In hindsight a statically typed language such as Go ¹ would have been more appropriate for a codebase maintained by nine developers in parallel. Python's dynamic typing means that type errors only surface at runtime, and several defects during integration could have been caught at compile time. Python also provides multiple interchangeable constructs for the same purpose, such as classes, dataclasses, named tuples, and plain dictionaries. When different team members made different choices between these, the result was inconsistent code. A statically typed language avoids this by offering fewer constructs and enforcing type correctness at compile time.
- **Ollama.** The client required that the language model must be run locally. Ollama runs in its own container, which keeps model execution separate from the backend. If a future deployment requires a cloud-hosted model instead, only the Ollama container (or the address to it) needs to be replaced.
- **Caddy.** Caddy was chosen as the reverse proxy because it provisions and renews TLS certificates automatically through Let's Encrypt ², with no additional tooling. The application currently runs behind the SURF Research Cloud portal, so TLS is not yet active, but the infrastructure is ready for a transition to a public deployment.
- **Podman and Quadlet.** Podman ³ with Systemd Quadlet ⁴ was chosen over Docker ⁵ for container management. Docker relies on a central daemon that manages all containers, effectively duplicating functionality that systemd already provides, namely process supervision, restart policies and logging. If the Docker daemon crashes, every container goes down with it. Podman eliminates this single point of failure (and reinvention of the wheel) by running each container as a regular process, allowing systemd to manage them directly. Quadlet extends this by defining containers in unit-file syntax, from which systemd generates a service for each one. This provides automatic startup, restart-on-failure and `journalctl` logging without additional tooling.

¹<https://go.dev/>

²<https://letsencrypt.org/>

³<https://podman.io/>

⁴<https://docs.podman.io/en/latest/markdown/podman-systemd.unit.5.html>

⁵<https://www.docker.com/>

5.2. Endpoints

Onno

The API is mounted behind a reverse proxy under the `"/api"` prefix. Routes are grouped into four routers; autocomplete, connections, entity, and pipeline plus two extra health endpoints. Every route follows the same three layers:

- A router validates the HTTP request and maps errors to status codes.
- A service holds the business logic.
- A repository executes Cypher against Neo4j.

The exception is the entity router, which issues two lookups directly through a request-scoped Neo4j session. wrapping those in a service layer would have added indirection without value.

Table 5.1: Endpoint overview

Route	Method	Purpose
/autocomplete	GET	Fulltext search for persons and organisations
/entity	GET	Display label for an entity (plain text)
/entity/info	GET	Profile card for a person (departments, ORCID, affiliations)
/entity/orginfo	GET	Minimal organisation info (name + internal flag)
/connections/entity	GET	Bulk load of all connection sections (initial page load)
/connections/collaborators	GET	Paginated co-authors
/connections/publications	GET	Paginated publications, with year/title filters
/connections/organizations	GET	Paginated affiliated/related organisations
/connections/members	GET	Paginated members (organisations only)
/connections/counts	GET	Capped item counts per section
/connections/highlights	GET	Top connections used as LLM prompt context
/connections/publication	GET	Full details for a single publication (by DOI)
/pipeline/{pipeline_type}	POST	Run an AI prompt and stream the result as SSE
/models	GET	Chat models available in Ollama (demo model picker)
/config	GET	Non-secret deployment flags read by the frontend
/health	GET	Shallow liveness probe
/ready	GET	Deep readiness probe (Neo4j + Ollama reachable)

5.2.1. Search and autocomplete

Search is a single endpoint: `"GET /autocomplete?query=...&limit=..."` that fills the search bar. It is intentionally forgiving, because users type do not always type in the entire name and might not put the first and last name in the same order as stored in the database ("`hooft gerard`" for "`Gerard 't Hooft`").

The flow is as followed: The service folds diacritics and normalises the query, splits it into keywords, and builds a Lucene query string. It then over-fetches candidates from Neo4j's fulltext indexes (person_name, org_name) up to a configurable cap, that clamped between 50 and 2000 rows because raw Lucene relevance is a poor final ranking. The candidates are re-scored in a single Cypher pass using a set of tunable weights (all exposed in the configuration file):

- exact / prefix / contains matches of the full query.
- prefix and word-boundary matches of individual tokens.
- a popularity term. $\log_{10}(1 + \text{output_count})$ for people. Member count for organisations.
- a flat boost for internal (Utrecht University) entities.

- small format bonuses (prefer "Surname, Given" and concise names).

After ranking, the service combines identical rows that come from source fragments (a.k.a when the same author appearing twice), keyed on the name, and recomposes the list. Over-fetching by a factor of five guarantees the dropdown still fills to the requested limit after both deduplication and regrouping.

Putting the ranking weights in configuration rather than hard-coding them was a direct response to real failures: common first names returned hundreds of index hits, and the desired person sometimes ranked below an arbitrary cutoff and vanished from the dropdown. Being able to tune the weights without a redeploy made search debuggable.

5.2.2. Entity connections

The "connections" endpoints back the panel that shows the raw graph data alongside the AI output, so users can always verify what the model was given. Each entity has up to four sections:

- collaborators: co-authors (persons only)
- publications :research outputs
- organizations : affiliations (for a person) or related organisations (for an organisation)
- members: people in the organisation (organisations only)

"GET /connections/entity" returns all these sections on one page load so it does not force four round trips. Internally the three active sections for a given entity type are fetched concurrently. Each subsequent "Load more" action hits the dedicated single-section endpoint (/connections/collaborators, /publications, etc.).

"GET /connections/counts" returns the counts per section. Counting used to be capped, but since new harvest scripts were made retrieval got fast enough to display the full count. Even for large organizations such as Utrecht University.

"GET /connections/highlights" is a special read used only as input to the AI prompts: it returns the most notable people and organisations connected to an entity so the pipeline can resolve names mentioned in generated text into clickable in-app links.

5.2.3. Internal vs. external entity split

The graph contains both Utrecht University (UU) entities from PURE and the large set of external co-authors and institutions harvested from OpenAlex. Per request by the client these should be split, since we do not have the full data available for external entities. If selected the results would be incomplete and the LLM might generate faulty responses. Every person and organisation carries an internal flag (is_uu_staff / is_uu in the graph, surfaced as is_internal in the API schemas). This flag is used in several places:

- Search: ranks internal entities above external ones as a hard group split: text relevance only orders entries within a group, so a slightly better-scoring external can never push a UU match out of view, while externals remain findable lower down.
- Connections: lists accept an optional internal query parameter (true = UU only, false = external only, omitted = both), letting the righthand panel load the internal and external groups independently.
- Profile cards: show a confirmed external affiliation for non-UU persons. This affiliation comes from OpenAlex enrichment during harvesting and is only shown when that enrichment produced a confirmed institution. For UU persons the frontend shows the UU name instead, driven by the is_uu flag rather than a graph-derived string.

- Some prompts search for external entities. For which this flag is ideal.

5.2.4. Pagination

All list endpoints use 'keyset' pagination rather than 'SKIP/OFFSET'. With offset pagination Neo4j must scan and discard every row, so deep pages get progressively slower. For organisations with thousands of members such as the University of Utrecht this is unacceptable. Keyset pagination instead carries the sort key and unique id of the last item on the page, and the next query's WHERE clause continues from exactly that position, keeping query time constant regardless of page depth. There are implementation details worth highlighting:

- Over-fetch by one: Every query asks for limit + 1 items. If the extra item comes back, another page exists and the cursor is emitted. The list is then trimmed back to the original limit. This avoids a separate 'count' query just to decide whether a "next" cursor is needed.
- Sort-aware cursors: People and organisations use a simple (sort_name, id) cursor. Publications need more, because they can be sorted by year or by title. The publication cursor therefore also carries a has_year flag and (for year sorts) the year value. The cursor value uses the internal sort key (lowercased, comma-stripped name), not the display name, so the keyset WHERE clause never skips rows that sort differently than they display.

5.2.5. Streaming response

The pipeline endpoint, "POST /pipeline/pipeline_type", is the only route that streams. AI answers can take tens of seconds to over a minute, and showing tokens as they are generated is essential to perceived performance by the end user.

Why Server-Sent Events (SSE) and not WebSockets. The requirement is strictly one-way: the server pushes tokens, the client only listens. SSE is a plain text/event-stream HTTP response, so it needs no connection upgrade, no separate protocol, and no persistent bidirectional socket to manage. WebSockets would add operational complexity (connection lifecycle, proxy configuration) for a features we do not need. SSE is very easy to implement using FastAPI's async StreamingResponse. It makes sure that the tokens are flushed as they arrive from Ollama without tying up a worker thread.

SSE frames are lines beginning with data: and terminated by a blank line. Each frame in this API carries a small JSON object whose single key identifies the event kind. The stream is ordered as followed:

```
data: {debug: {...}} - only when PIPELINE_DEBUG is on
data: {sources: [{"n": 1, "title": "..."}]} - citation source list
data: {entity_mentions: {...}} - names for clickable links
data: {thinking: "..."} - repeated , LLM chain of thought (if enabled for gemma)
data: {token: "..."} - repeated, a part off the LLM answer
data: {phase_change: true} - to indicate the short summary is being generated
data: {token: "..."} - repeated, now the summary data
data: [DONE] - indicates that the generation has finished
```

The design choices behind this structure:

- Sources and entity mentions are sent up front, as their own frames, before any token. This means the references box and the name-to-link mapping can be displayed reliably even if the model forgets to append a trailing sources block or phrases a name wrongly and the frontend does not have to parse them out of the text.
- We put the reasoning in a separate frame from the answer so these two can easily be split in the frontend, allowing for a collapsible panel, clearing up space.

- [DONE] was made to be a separate item, so the finalization can be indicated reliably. Not just when the LLM is done generating an answer, but also when an error occurs. This is essential so the SSE connection can be closed gracefully and resources on the server can be cleared.

5.2.6. Abuse protection

The API itself has no authentication (Per the requirements of the client). The protections implemented are therefore aimed at a single abuser that would try to exhaust the single GPU or flood the process. Three techniques were used to mitigate this:

- Rate limiting: Requests are bucketed by client IP over a rolling 60-second window. Cheap reads get a generous budget (240/min by default). The GPU-expensive pipeline route gets a tight one (10/min), because each pipeline call occupies the GPU for tens of seconds. Exceeding a budget returns 429 with a 'Retry-After' header. State is in-process (one worker), and idle clients are pruned periodically so the map cannot grow unbounded.
- Request-body size cap: Every legitimate request body is a small JSON document, so a content length above 64 KB is rejected with 413 before the body is read. Parsing a large body would burn CPU and memory for nothing.
- Global pipeline concurrency cap: Independent of the per-client rate limit, a process-wide counter limits how many pipeline streams run at once (3 by default). When the cap is reached the API refuses immediately with 429 rather than queueing.

All three limits are configurable, and setting a limit to 0 disables that bucket.

5.3. Database

Onno

5.3.1. Node architecture

There are four node types. The application creates the indexes it needs at startup, but the nodes themselves are produced by the offline build.

Person: after the data cleanup there should be one node per real human (UU staff and external co-authors alike) with the following attributes.

Table 5.2: Person node attributes

Property	Notes
id	Canonical internal id (a Pure uuid or an OpenAlex author id)
name	Canonical display name
oa_display_name	OpenAlex display name, kept when it differs (shown as "also known as")
search_ascii	Diacritic-folded name, used for fulltext search
orcid	ORCID identifier (single)
scopus_id, isni, dai	Other external person identifiers
oa_ids	OpenAlex author ids merged into this person (0..n)
pure_uuid	Present iff the person exists in Pure
is_uu_staff	True for Pure-backed UU employees; false for external co-authors
n_outputs	Number of authored outputs (used for popularity ranking)
sources	Provenance, e.g. pure,openalex
search_hidden	Hides letterless placeholder names from search

ResearchOutput: These are all the publications. The most notable part is that these nodes also store a vector index, allowing for semantic search.

Organisation: All harvested universities, companies, funders, facilities, and internal UU units.

Table 5.3: ResearchOutput node attributes

Property	Notes
id	Canonical id: the DOI when present, else a synthetic pure:/oa:/rsd: key
doi	Normalised DOI; the primary deduplication key
title	Cleaned title
abstract	Cleaned abstract text (when available)
has_abstract	Whether an abstract is present (~ 59% of outputs; a retrievability flag)
embedding	1024-dim vector (title + abstract + topics + venue)
year, type, language	Publication year; unified 16-value type; language
is_research_output	Unified flag; the default filter for rankings and retrieval
venue, is_oa, oa_status	Journal/venue and open-access status
pure_uuid, oa_key	Source ids kept for traceability
sources	Provenance

Table 5.4: Organisation node attributes

Property	Notes
id	ror:<ror> when a ROR id exists, else puorg:pure-uuid for internal units
ror	ROR id, the canonical join key across Pure, OpenAlex, and ROR
name	Display name
search_ascii	Diacritic-folded name for search
org_type	education / company / funder / facility
country, country_code, city	Location
is_uu	True for UU and its internal units
sources	Provenance

Topic: OpenAlex topic categories, with level distinguishing topic / subfield / field / domain and parent_id linking up the tree.

Table 5.5: Topic node attributes

Property	Notes
id	OpenAlex topic/subfield/field/domain id
name	Topic name
level	topic, subfield, field, or domain
parent_id	The id one level up

5.3.2. Schema of possible connections

All relationships are typed and stored once. Which is one of the major improvements implemented in the last database revision, where previously the all connections were stored both ways. The new method save on data and still allows for two-way traversal through the graph.

The CO_AUTHORED connection deserves a quick note: It is only save once per author pair and save a traversal step when searching for “top collaborators” compared to the previous implementation. Consortium papers (more than 50 authors) are excluded however, since they would otherwise create enormous numbers of weak, misleading collaboration edges. Next, the PART_OF hierarchy uses the same single stored edge for both directions: walking up answers “which faculty/university does this department belong to”, walking down answers “which sub-units sit under this organisation”. This is for example used to find experts within a faculty sub-unit.

The :PART_OF hierarchy uses the same single stored edge for both directions: walking up answers “which faculty/university does this department belong to”, walking down answers “which sub-units

Table 5.6: Topic node attributes
P=Person, O=Organisation, RO=ResearchOutput, T=Topic.

Edge	From	To	Properties	Used for
AUTHORED	P	RO	seq, corresponding, position	a P's outputs / a paper's authors
CO_AUTHORED	P	P	n_outputs, last_year	finding top collaborators
AFFILIATED_WITH	P	O	from, to, current, source, n_works	"works at" / "members of this unit"
PART_OF	O	O	-	org hierarchy, both up and down
HAS_TOPIC	RO	T	score	"papers on topic" / "topics of paper"
SUBFIELD_OF	T	T	-	the topic tree

sit under this organisation" (used, for example, to find experts within a faculty's sub-units).

5.3.3. Data harvesting

The graph is built offline from harvested data using a medallion pattern. Where each tier can always be rebuilt from the one below it. We have three tiers, Bronze , Silver and Gold. The Neo4j database is just a loaded copy of the gold tier. In short:

- Bronze: What the source said
- Silver: what we understood
- Gold: What the app uses

Table 5.7: Topic node attributes

Layer	Contents	Storage
Bronze	Raw API responses, byte-for-byte, plus cached embeddings. Append-only, never edited.	bronze.sqlite (zstd-compressed payloads)
Silver	Cleaned, normalised intermediate tables, then deduplicated/clustered.	staging/*.parquet
Gold	Final node/edge tables in the exact shape the app needs.	gold/*.parquet to comma separated values

The bronze layer should be the source of truth. The harvest does nothing but capture raw payloads into bronze and at every later stage it re-parses bronze with its own stricter extractors. This means that if there is a bug in the parser, it can be fixed and the entire graph rebuilt without the need to re-harvest. Each harvested HTTP response is cached by a hash, so a re-run only fetches what bronze does not already have. Embeddings are also cached, so a rebuild with unchanged abstracts uses zero GPU.

5.3.3.1. Data sources

ROR and ORCID are *enrichment* sources: they are fetched after the first extraction pass, driven by the identifier lists that the primary sources surfaced, so each record is fetched exactly once and reused forever.

5.3.3.2. Harvest and build steps

The full rebuild runs as an ordered chain (silver/run_all.py after a harvest):

Table 5.8: Harvest sources

Sources	Provided data
Pure	Research outputs, datasets, persons, and the organisation hierarchy of the UU
OpenAlex	Works by institution ROR, with per-work authors, affiliations, and topics
RSD (Research Software Directory)	Research software entries
Yoda	Datasets via the DataCite OAI-PMH feed
UU Staff Pages	Staff profiles (optional)
ROR	Organisation records for every ROR id seen in the data (enrichment)
ORCID	Person records for every ORCID seen in the data (enrichment)

1. **Harvest:** All harvesters run in parallel, each fetching through a shared HTTP layer (retries, rate limits) that transparently captures every response into bronze. No database is written.
2. **Extract:** Bronze payloads are parsed by strict per-source extractors into normalised staging tables (one row per source record).
3. **Fetch enrichments:** ROR and ORCID payloads are fetched for the identifiers found in staging.
4. **Cluster persons:** Pure persons and OpenAlex authors are merged into real people, producing an 'entity_id' to 'canonical_id' mapping.
5. **Build gold tables:** The deduplicated, merged node and edge tables are assembled, plus the per-output text to embed.
6. **Validate:** A dry-run asserts data integrity (no duplicate ids or DOIs, no orphan edges, no title-less outputs, no nameless persons) and prints coverage stats and sample merged profiles to the console.
7. **Embed:** The gold output texts are embedded (delta-only via the bronze embed cache) into 1024-dim vectors.
8. **Export and import:** Gold is exported as neo4j-admin import CSVs and loaded into a staging Neo4j, which is then dumped and shipped to production.

It should be noted that a lot of effort has to go into cleaning and merging the data. If properly done, The net result is a graph that is smaller and denser with meaningful signal than the original. In our case it is in the order of 0.8M nodes instead of 2.8M.

5.4. Pipelines

Ivan

The key component of Research AI is the ability to feed information into a Large Language Model (LLM) and obtain an output. Within the code base, the full journey from a user clicking a prompt button to the LLM returning an answer is called a pipeline. Each prompt has its own pipeline, specialised in one specific task. The pipelines must:

- Collect evidence: from the knowledge graph (publications, affiliations, collaborators) and, when needed, from semantic search over publication text.
- Build a prompt: for the large language model, containing all relevant information and the right instructions, including numbered source blocks for citations.
- Stream the answer: the response is streamed back to the browser in small blocks, and for some prompts a short bullet-point summary is generated.

The user does not see these internal steps. They see a list or narrative containing the information they requested, grounded in proper citations and drawn only from the system's own data. When the user submits a request, the backend receives the chosen prompt type, the selected person

or organisation, and optional filters (such as topic, year range and language), and then runs the matching pipeline from start to finish.

5.4.1. Object-oriented approach

The current design is built around small, reusable building blocks. Because each pipeline contains certain steps that are also needed by other pipelines, each of these steps has been given its own function, forming building blocks that can be stacked to create a working pipeline. Each block has a single function, such as fetching publications, attaching abstracts, fitting text within a token budget, assembling the final prompt, or streaming the LLM response. Blocks with similar functions, or that always follow one another, are grouped into a series of letters.

For example, the Layman pipeline uses, in this order, A, B, C, G, D, E and F, whereas the top-collaborations pipeline uses only D, E and F. Depending on the functionality a pipeline needs, it can select from the letter blocks.

Table 5.9: Letter series and their roles in the pipeline.

Series	Role in plain terms
A	Publications: load from the graph, clean up duplicates, fetch and attach abstracts
B	Graph context: affiliations, top collaborators, organisation members, and related organisations, rendered as readable sections
C	Retrieval strategy: choose graph-first vs semantic search; optional “hypothetical abstract” (HyDE) to improve search; turn papers into evidence blocks
D	Budget: decide how much text fits in the model’s context window; trim ranked lists or publication lists
E	Prompt assembly: combine entity description, instructions, language, and evidence sections
F	Model call: pick system instructions, sampling settings, and stream the answer (plus optional summary phase)
G	Topic and scope: filter publications by subject, restrict to “external” partners, aggregate counts
T	Topic mining: extract research themes from many papers (used for “top research topics”)

5.4.1.1. Two kinds of answers

There are two kinds of answer an LLM can give, depending on which pipeline is used (see Table 5.12). There is also one special kind of prompt. The system distinguishes between the following two:

- **Narrative pipelines:** these produce a narrative answer, such as a summary of the research performed or a researcher’s CV. The model is instructed to cite numbered documents and to stay close to the supplied evidence.
- **Ranked-list pipelines:** these produce an ordered list (top five collaborators, top organisations, top themes). The ranking and counts are largely fixed before the model runs. The job of the LLM is only to present that list clearly and in the correct language, without reordering or inventing entries.

5.4.1.2. Narrative pipelines

The Layman summary (see Figure 5.1) is one of the richest flows, with several decision points that are implemented differently depending on the situation. When the request arrives, it is accompa-

Table 5.10: Pipeline features overview

Pipeline	Answer kind	Main idea
Layman summary	Synthesis	Graph context + smart paper selection (graph, search, or topic filter)
NWO-style CV	Synthesis	Same as layman; different section instructions
Recent publications	Synthesis	Chronological papers from graph
Recent publications by topic	Synthesis	Search by topic; fallback if no hits
Output matching on subject	Synthesis	All papers relevant to a theme
Top collaborating organisations	Ranked list	Graph ranking by shared publications or members
Top collaborators	Ranked list	Graph ranking of co-authors
Top research topics	Ranked list	Mine themes from papers, then present top five
Top external collaborations on subject	Ranked list	Topic filter + external scope + counts
External collaboration partners	Ranked list	External scope; graph shared-pub rank (org) or vector topic match (person)
Topic experts	Ranked list (special)	Vector search + graph join; output generated without chat model

nied by three arguments: the entity for which the summary must be generated, the output language, and optionally a specific topic to focus on.

If a topic is given, the filters in series G are used to filter all publications and output only those that align with the given topic. If no publications match that topic, the system instructs the LLM to output an error message, accompanied by the five most recent publications, so that the user gets an idea of the kind of publications this entity produces.

If no topic is given, a second decision point considers the number of publications related to the entity. If there are fewer than 60 publications, all titles and abstracts are included, as these fit within the LLM context window. If there are more than 60 publications, a filter is needed to reduce this number to only those that fall within the entity's main research focus. This is done by the T functions, which return the five most-researched subjects and their related publications. If this does not work (because, in rare cases, an entity has no clear main research focus), the system falls back on the 60 most recent papers.

At this point there is a list of DOIs, to which the titles and abstracts must be added. Evidence blocks are then created from this information: the details of each publication are stored together and assigned a reference number that the LLM can refer back to in the text. This is then merged with additional information about the entity and its most important research subjects. A final check verifies that the input is still within the LLM's context bounds; the final instruction text is then written and sent, and the answer is streamed back.

5.4.1.3. *Ranked-list pipelines*

Ranked pipelines (see Figure 5.2) are simpler than the narrative pipelines, as they do not need to summarise the publications. For questions such as “Who are the top collaborating organisations?”, the answer is already a sorted list, and the LLM is only needed to present it as readable text. For this, the system uses a shorter pipeline:

- Check the budget: the model can only read a limited amount of text at once, so the system determines how many list items it can fit.
- Get the list from the database: this output is already ranked.
- Turn each row into a simple text line.
- Trim the list if it exceeds the LLM budget constraints, although this is usually not necessary.

- Add instructions and send the data to the LLM.

5.4.1.4. *Special pipeline: Topic experts*

The topic-experts pipeline is a third, special kind of pipeline. It does not follow the standard ranked-list path or the narrative path. Instead, it combines retrieval and graph reasoning to rank people, and then assembles the answer in the code; no AI model writes the final text. This pipeline operates within a (large) organisation: a topic is fed in, and the pipeline finds the person within the organisation who is most familiar with it. It works as follows:

- **Normalise** the topic keyword.
- **Seed HyDE** with recent publication titles from the organisation: providing recent titles allows domain vocabulary to be extracted, which helps in finding the best results.
- **HyDE + vector search**: an LLM writes a short hypothetical abstract, which is embedded and matched against the organisation's publications.
- **Graph join**: the retrieved papers are linked to the co-authors from within the given organisation.
- **Rank**: authors are ordered by their best publication similarity score. The top experts are retained, together with up to three evidence papers.
- **Stream** the answer to the user with sources and person links, without any LLM generation.

5.4.2. **Shared infrastructure across all pipelines**

Several concepts tie every pipeline together without duplicating logic.

1. **Registry**: all pipeline requests arrive at a central place, where the server looks up the pipeline name and runs the corresponding pipeline object. Dependencies such as graph access and the AI service are injected when the registry is built.
2. **Run state**: each request has a "mutable state object" that is used while the prompt is built. This object holds information such as debug metadata (retrieval mode, timings, prompts), source mentions, and optional entity mentions. The latter two make it possible to render sources and person/organisation names as clickable links.
3. **Base pipeline behaviour**: each pipeline shares the same final steps, such as calling the LLM, streaming its response, and adding an optional summary. This is achieved by inheriting the "common streaming path" object.

5.4.3. **Helper functions**

Not every repeated action has its own class. There are also helper functions that do not interact directly with the database or the LLM, but are used for several purposes such as formatting, counting tokens, and joining sequences. There are different kinds of helper function:

1. **Orchestration helpers**: these run a fixed chain of building blocks that many pipelines need. Some examples are:
 - (a) loading a wide publication pool and then filtering by topic;
 - (b) fetching publications within a year-window filter;
 - (c) attaching abstracts to a set of papers;
 - (d) finalising a ranked-list prompt by computing the budget, fitting lines, wrapping the section, and building the message.
2. **Formatting and budget helpers**: these use pure logic to, for example, turn a publication into

prompt lines, number document blocks, estimate token usage, and drop the lowest-priority content until the text fits.

3. **Topic-mining helpers:** split publications into batches, parse model JSON, merge synonymous topic labels, count and rank, and format lines.

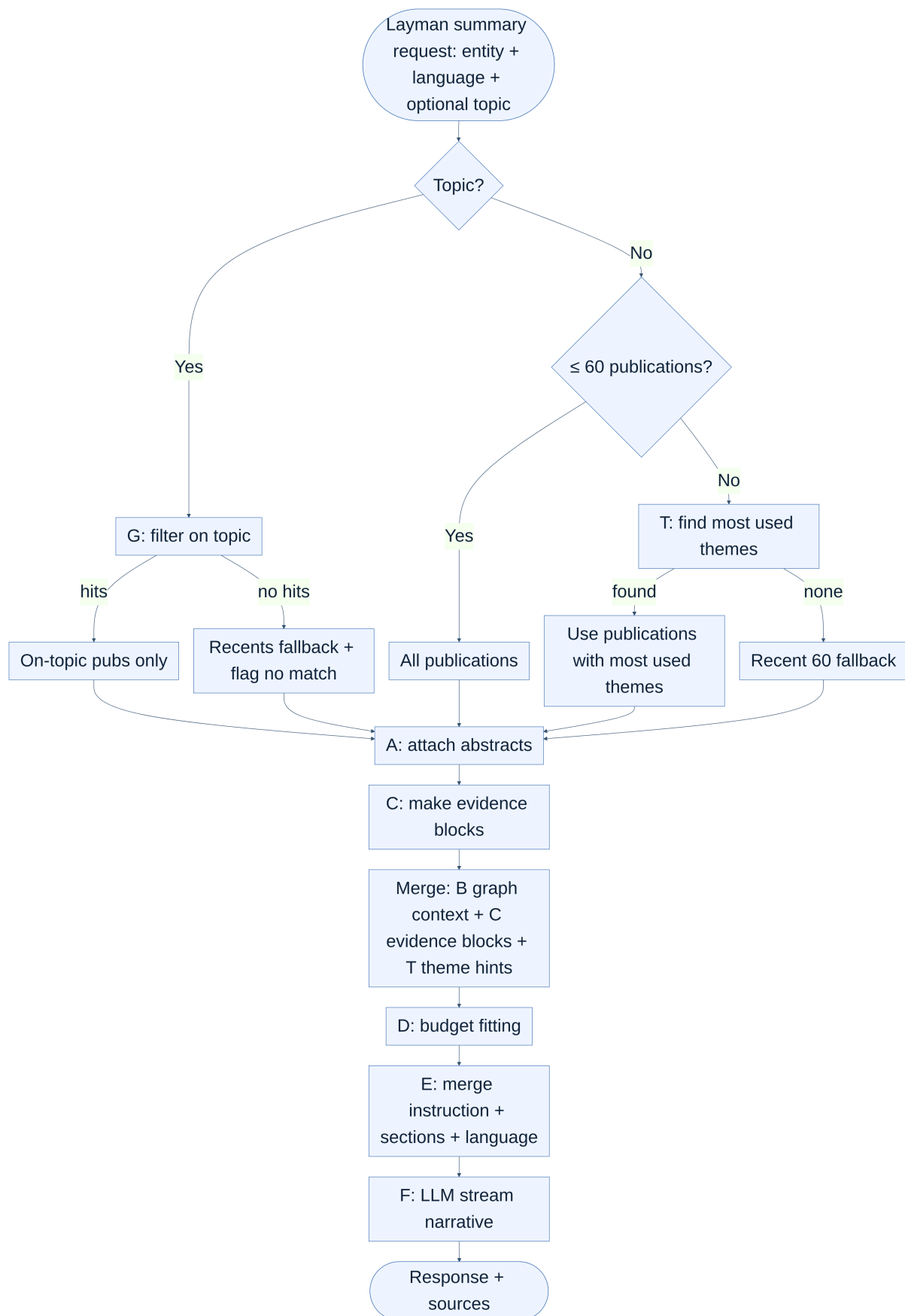


Figure 5.1: Layman summary pipeline

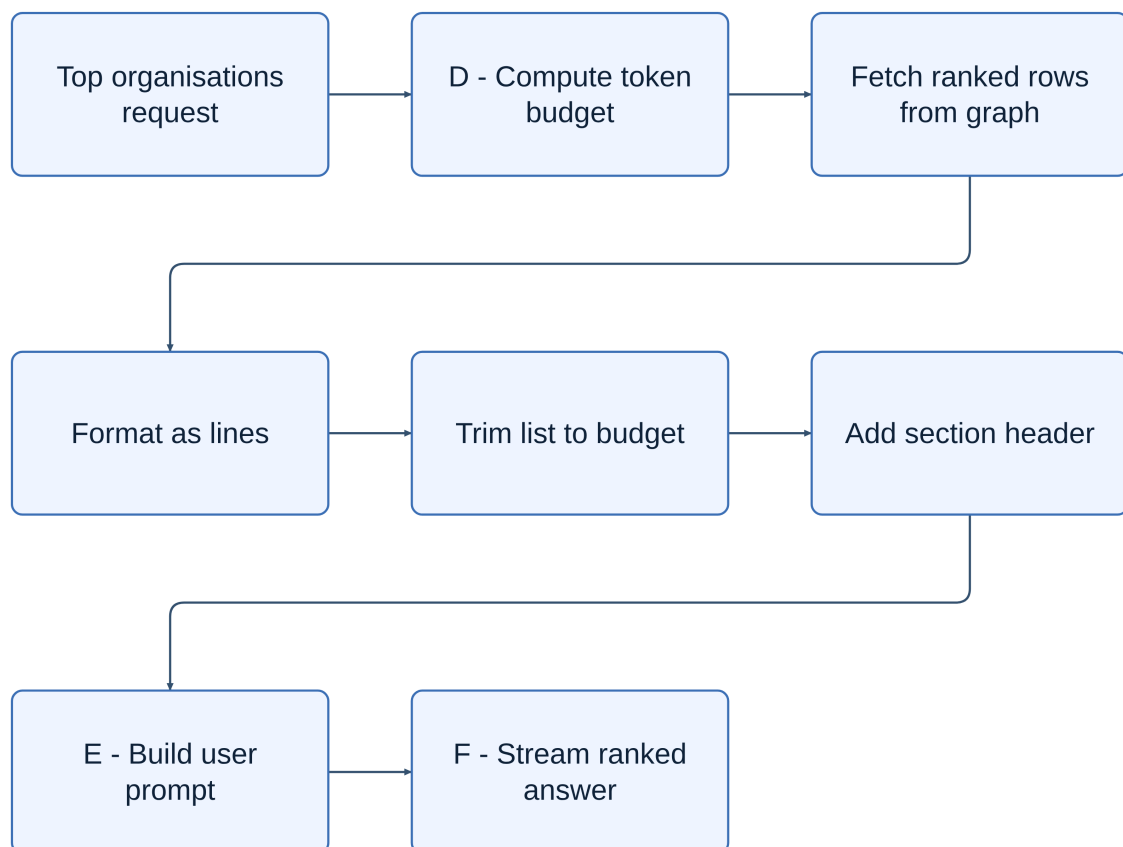


Figure 5.2: The top organisations pipeline

5.5. Large Language Model

Onno

The system runs all models locally through Ollama, as required by the client. There is a single NVIDIA A10 GPU (24 GB VRAM) available on the production server, which was an important constraint for selecting a model. The following functional requirements also needed to be taken into account:

- Response time under 20 seconds
- Low hallucination (Truthfulness as the first priority)
- Inline citations
- Multilingual handling

Three models run side by side and together fit the 24 GB card.

Table 5.11: AI model overview

Role	Model	Notes
Chat / generation	<code>gemma4:26b</code>	Mixture-of-experts; supports “thinking”; product default
Retrieval helper (HyDE)	<code>qwen3:1.7b</code>	Tiny, co-resident on the GPU
Embeddings	<code>qwen3-embedding:0.6b</code>	1024-dim, cosine; must match the stored vectors

Chat model: Gemma 4 26B (MoE). This replaced an earlier `command-r:7b`. The mixture-of-experts design is the key reason: only a fraction of the 26B parameters are active per token, so it generates at small-model speed while answering at large-model quality, meeting the same sub-20 second target the 7b model was chosen for. It also supports a “thinking” mode, which is exposed as a UI toggle and improves the response quality at the cost of speed. While `command-r` was initially chosen for its training on citations, the citation rules that have been engineered by the team have proven to be reliable enough in combination with Gemma to warrant the replacement. The context window is set to 16,384 tokens in the environment file, which has been validated on the A10. It must not be raised without re-running the context benchmark.

HyDE model: Qwen3 1.7B. Retrieval uses HyDE (hypothetical document embeddings). This model writes a short hypothetical ~100 word abstract whose embedding replaces the raw query in vector search. That output is consumed only by the embedder and never shown to the end user. The tiny 1.7B model co-resides with the other models. Its thinking mode is force-disabled so reasoning text never pollutes the embedding query. A dedicated `HYDE_MODEL` is set explicitly (rather than defaulting to the chat model) so that swapping the chat model cannot silently change retrieval behavior.

Embedded model: Qwen3-Embedding 0.6B. This is the one model that is not a free choice at deploy time. The graph’s stored vectors are produced offline by the harvest pipeline, and query embeddings are only comparable to document embeddings from the same model: deploying a different embedder produces vectors of the same dimensionality in a different space, so vector search silently returns meaningless results with no error. It was chosen for comparable multilingual retrieval quality at a smaller footprint than the previous “snowflake-arctic-embed2” embedder, and for sharing a model family with the HyDE model. Changing it requires re-embedding the whole graph (2.5 hours of GPU time) and re-importing the graph.

5.6. File structure

Onno

The backend lives under ‘api’. The runtime application is ‘api/app’, organised by technical concern so the layout mirrors the layered architecture. Offline data scripts live under ‘api/app/scripts’, and

all the tests under 'api/tests/"/>.

Table 5.12: Backend folder structure overview

Path	Responsibility
app/main.py	Application entry point: lifespan, middleware, router registration, /health and /ready
app/config.py	Environment-driven settings with fail-fast validation; LLM sampling presets as constants
app/dependencies.py	Shared FastAPI dependency type aliases
app/exceptions.py	Domain exception hierarchy, mapped to HTTP status codes in routers
app/prompts.py	Pipeline name constants and prompt templates
app/routers/	HTTP handlers: autocomplete, connections, entity, pipeline
app/services/	Business logic and the abstract service interfaces (interfaces.py)
app/repositories/	Neo4j driver singleton, repository classes, .cypher query files, dynamic query builders
app/schemas/	Pydantic request/response models (ai, connections, entities)
app/utils/	Stateless helpers: formatters, pagination cursors, TTL cache, Lucene escaping, token budgeting, abuse protection
app/pipelines/	Pipeline registry, step base classes, and the concrete prompt pipelines (covered in the main report)
app/scripts/	Offline data harvesting and graph build; never imported by the running server
tests/	unit/, integration/, system/, plus per-module behaviour docs under tests/docs/

Two structural conventions are worth calling out:

- Cyphers are stored in a folder within 'repositories'. Complex queries are stored as individual ".cypher" files in 'repositories/cypher/' for readability. While queries that only vary by sort order or pagination cursor are assembled by dynamic builders in queries.py to avoid maintaining many near-identical copies. A naming convention maps Cypher's camelCase parameters (\$entityId) to Python's snake_case (entity_id).
- The offline harvest scripts carry no request-path dependencies. They are a separate program that produces the graph; the running API only ever reads from the graph. This separation is what makes the graph a disposable, rebuildable artifact.

5.7. Configuration and Environment file

Onno

All configuration comes from environment variables, read once at startup into a single Pydantic BaseSettings object (config.py). To make configuration safe the "fail fast" principle is applied on missing required values. Four variables have no default:

REMOTE_NEO4J_URL, REMOTE_NEO4J_USER, REMOTE_NEO4J_PASS, and AI_SERVICE_URL. If any is absent the application refuses to start, rather than failing confusingly on the first request.

Everything else has a sensible default and is overridable: model names, the chat context window and token budget, embedding dimensionality and GPU layers, cache sizes and TTLs (for connection results, embeddings, HyDE outputs, and topic extraction), the vector similarity floor, the fourteen autocomplete ranking weights, the abuse-protection budgets and logging level.

There are also a few deployment flags that visible to the frontend with 'GET /config': DEMO_MODE, which exposes tester controls such as the model picker and thinking toggle. And HARVESTING, which switches the UI to an "offline for harvesting" banner while the graph is being rebuilt.

There are multiple distinct LLM presets (temperature, top-k, top-p, and repeat penalty), with

a different preset for synthesis, factual-list, factual-prose, and HyDE generation. They are kept as module-level constants (not env-driven) because they are internal tuning knobs rather than deployment-varying configuration. For example, factual-list generation runs at temperature 0.0 for determinism, while factual prose uses 0.1 specifically to stop the model from pattern-completing plausible-but-fake publication entries.

Deployments are configured through an environment file in 'kube/*.env'. The file also configures the harvest pipeline, which reads the same kind of variables (Pure credentials, the home-institution ROR, the publication-year range, OpenAlex and ORCID settings). This single-source-of-configuration approach is what makes the system portable. Another institution could adopt it by editing the environment file. Pointing PURE_URL at their own PURE system, HARVEST_ORG_ROR at their own institution, and AI_SERVICE_URL at their own server running Ollama. All without touching application code.

5.8. Frontend Implementation

Thijmen

This section describes the implementation of the Frontend, that realizes the architecture of the frontend as described in Section 4.3..

5.8.1. Technology Stack

The frontend is built on the following technologies:

- **Core framework:** Building on the required React framework, we opted for compiler-enabled optimizations via the `@vitejs/plugin-react`⁶ and the `babel-plugin-react-compiler`⁷ packages to maximize rendering performance.
- **Language:** TypeScript⁸ provides static type safety across the entire codebase. We utilize strict typing for all component props, custom hooks, and API contracts, ensuring that data structures are validated at compile time.
- **Routing:** The `react-router-dom`⁹ library handles client-side routing. We defined a `createBrowserRouter` configuration with three top-level routes: the home page (`/`), the entity detail page (`/:type/:id`), and the documentation page (`/docs`).
- **Internationalization:** We use `i18next`¹⁰ to manage the English and Dutch locales. To prevent untranslated text flashes, the application loads the `i18n` configuration synchronously in `main.tsx` before the React tree mounts. The system persists the user's language preference in `localStorage` and dynamically updates the HTML `lang` attribute. Two locale files (`en.json` and `nl.json`) cover all user-facing strings, including prompt labels, error messages, and accessibility attributes.
- **Networking and streaming:** The application utilizes `axios`¹¹ with a singleton instance configured with a global error interceptor for standard HTTP communication. For the one-way LLM token stream, it directly employs the native `fetch()` API with `ReadableStream`.
- **Rendering and UI:** The `react-markdown`¹² package handles markdown rendering, utilizing its component override mechanism to inject entity links and citation cross-references. Furthermore, `react-hot-toast`¹³ manages notifications, and `react-error-boundary`¹⁴ catches com-

⁶<https://www.npmjs.com/package/@vitejs/plugin-react>

⁷<https://www.npmjs.com/package/babel-plugin-react-compiler>

⁸<https://www.typescriptlang.org/>

⁹<https://reactrouter.com/>

¹⁰<https://www.i18next.com/>

¹¹<https://axios-http.com/>

¹²<https://github.com/remarkjs/react-markdown>

¹³<https://react-hot-toast.com/>

¹⁴<https://www.npmjs.com/package/react-error-boundary>

ponent tree failures.

5.8.2. Component Structure

The components that populate the four layers as described in Section 4.3. are organized as follows. All routes are wrapped by a shared `AppLayout` shell that renders the active page via React Router's `Outlet`. The router itself dispatches to one of three primary page components: `Home`, `Entity`, or `DocumentationPage`.

- **Page-level components:**

- `Home (/)`: Serves as the landing page, centralizing the entity search functionality.
- `Entity (/:type/:id)`: Operates as the primary workspace for a specific person or organization. It coordinates AI prompt generation, streamed response history, breadcrumb navigation, and the connections side panel.
- `DocumentationPage (/docs)`: Delivers a standalone end-user documentation view, rendering localized Markdown with in-app routing for internal links.

- **Feature-specific composite components:**

- **Search and navigation:** The `SearchBar` handles autocomplete with keyboard navigation and breadcrumb seeding, while the `BreadcrumbTrail` maintains the navigation history synchronized with the internal breadcrumb store.
- **AI generation and output:** The `PromptTray`, `PromptCategoryModal`, and `PromptButton` components construct a localized, entity-type-aware prompt catalog. The `AiOutputHistory` and `AiOutputBox` manage streamed Markdown rendering, entity linkification, and citation parsing. Furthermore, the `ReferencesBox` separates cited and uncited source lists, and a `DebugPanel` exposes an optional RAG debugging interface (gated by the `VITE_DEBUG` environment variable).
- **Connections panel:** The `RightPanel` orchestrates paginated people, publications, and organizations sections (incorporating debounced filters and eager prefetching) within a responsive `SlidingPanel` container. It delegates rendering to section composites (e.g., `PeopleSection`, `PublicationsSection`) and their supporting subcomponents, such as the `PersonInfoCard`, `PublicationItem`, and various panel state loading skeletons.
- **Publication details:** The `PublicationDetailModal` fetches and displays publication metadata within an isolated modal dialog.

- **Primitive components:**

- `SelectControl`: Provides a generic, accessible select interface with detached and floating dropdown variants, utilized throughout the application for sorting, filtering, and language selection.
- `ExpandableTruncatedText`: Manages reusable truncated text blocks with expand and collapse behaviors, implemented across search results, connection lists, and publication titles.

- **Layout components:**

- `AppLayout`: Functions as the root layout shell, wrapping all routed pages via the React Router `Outlet`.
- `Header`: Renders the site header, containing the application logo, an optional center slot (typically utilized for the `SearchBar`), and right-aligned user controls.
- `LanguageButtons`: Integrates the `SelectControl` with the `i18n` configuration to provide a seamless language switcher.
- `DocsSiteLink`: Directs users to the documentation, automatically hiding itself when the active route matches `/docs`.

- `CopyrightFooter`: Injects a consistent, site-wide copyright notice at the bottom of the viewport.

5.8.3. File Structure

The file structure of the frontend follows a flat, concern-based organization. Table 5.13 details the responsibility of each primary directory. At the root level, standard Vite configuration files

Directory / File	Architectural Responsibility
<code>main.tsx</code>	
<code>pages/</code>	Acts as the composition root for each route in the application (<code>home/</code> , <code>entity/</code> , <code>documentation/</code>).
<code>components/</code>	Houses reusable UI building blocks, grouped by feature. Complex features (e.g., <code>RightPanel/</code>) are further subdivided into <code>parts/</code> for presentation and <code>hooks/</code> for localized data orchestration consumed only within that module.
<code>hooks/</code>	Encapsulates global stateful logic and side effects shared across multiple pages (e.g., <code>useStreamingResponse</code> , <code>useBreadcrumb</code> , <code>fetchEntity</code>).
<code>services/</code>	Contains non-UI infrastructure modules that interact with persistent storage (e.g., IndexedDB caching in <code>llmTurnCache</code>) and legacy data migration helpers.
<code>types/</code>	Consolidates shared TypeScript interfaces into a single file (<code>types/types.ts</code>), divided into backend responses, pagination states, and internal UI models.
<code>locales/, assets/</code>	Hold the static i18n translation JSON files (<code>en.json</code> , <code>nl.json</code>) and shared SVG icon components (<code>icons.tsx</code>), respectively.
<code>api.ts</code>	Provides the globally accessible Axios singleton and typed REST client functions (<code>autocomplete</code> , <code>connections</code> , <code>entity lookups</code>). A global error interceptor delegates failures to <code>toast.ts</code> .
<code>i18n.ts, toast.ts</code>	Root-level utilities for internationalization bootstrap and global error notifications.

Table 5.13: Overview of the `src/` directory structure and corresponding separation of concerns.

(`vite.config.ts`, `tsconfig.json`, `eslint.config.js`) and the `Containerfile` govern the build process.

5.9. Deployment

Author: Jelmer | Reviewer: Timon

5.9.1. Deployment Workflow

There is no continuous integration or automated deployment pipeline. To deploy a new version, a developer connects to the server via SSH, pulls the latest changes from the Git repository and runs `make deploy`. This command rebuilds the container images for the API and frontend from their respective `Containerfiles`, installs the Quadlet unit files and environment configuration into `/etc/containers/systemd/` and `/etc/research-ai/`, creates any missing Podman volumes and networks, runs `systemctl daemon-reload` to regenerate the service units, restarts all services and pulls the required AI models into Ollama. The entire process is defined in the `Makefile`, so deploying a

new version requires only two commands. During normal development, the full deploy is not necessary as developers use the local setup described in Section 5.9.3..

The Ricgraph harvesting process and the enrichment scripts are triggered manually on the server. Ricgraph pulls metadata from external sources such as Pure, OpenAlex, the Research Software Directory, Yoda and the UU Staff Pages. After harvesting, the enrichment scripts fetch abstracts and affiliations from OpenAlex and the Pure API and generate vector embeddings via Ollama. These processes are run infrequently and do not need to be repeated unless the underlying data sources have been updated.

Logs for all services are accessible through `journalctl`, since each container runs as a `systemd` service. The Makefile provides convenience targets such as `make logs` to tail the output of the API, frontend and Ricgraph services simultaneously.

5.9.2. Container Configuration

All containers are managed as `systemd` services via Podman Quadlet and communicate over an internal Podman network (10.89.0.0/24) using container hostnames.

Each container is defined in a `.container` Quadlet file that specifies the image, network, volumes, environment variables and resource constraints. Figure 5.3 shows a simplified example for the API container. When the file is placed in `/etc/containers/systemd/` and `systemctl daemon-reload` is run, `systemd` generates a corresponding service unit, which can then be managed with `systemctl` and `journalctl`.

```
[Container]
ContainerName=api
Image=research-ai-api:latest
Network=research-ai.network
Environment=AI_SERVICE_URL=http://ollama:11434
Environment=REMOTE_NEO4J_URL=bolt://neo4j:7687
PublishPort=8000:8000

[Service]
Restart=on-failure

[Install]
WantedBy=default.target
```

Figure 5.3: Simplified Quadlet file for the API container. `Systemd` generates a service unit from this definition, allowing standard service management with `systemctl`.

The frontend container uses a two-stage build. The first stage uses a Node.js image to install dependencies and compile the React application with Vite¹⁵. The second stage copies the resulting static files into a Caddy image, which serves them at runtime. This keeps the production image small because build tools and `node_modules` are not included in the final image.

5.9.3. Development environment

In the development environment, the frontend and API run locally in a Podman pod with hot-reload enabled (Figure 5.4). The source directories are mounted into the containers as volumes, so file changes are picked up automatically by Vite (frontend) and `uvicorn` (API) without requiring a con-

¹⁵<https://vite.dev/>

tainer rebuild. Neo4j, Ollama and the Ricgraph Explorer web interface remain on the production server and are reached via an SSH tunnel with port forwarding. This means developers do not need a GPU or a local Neo4j database to work on the application.

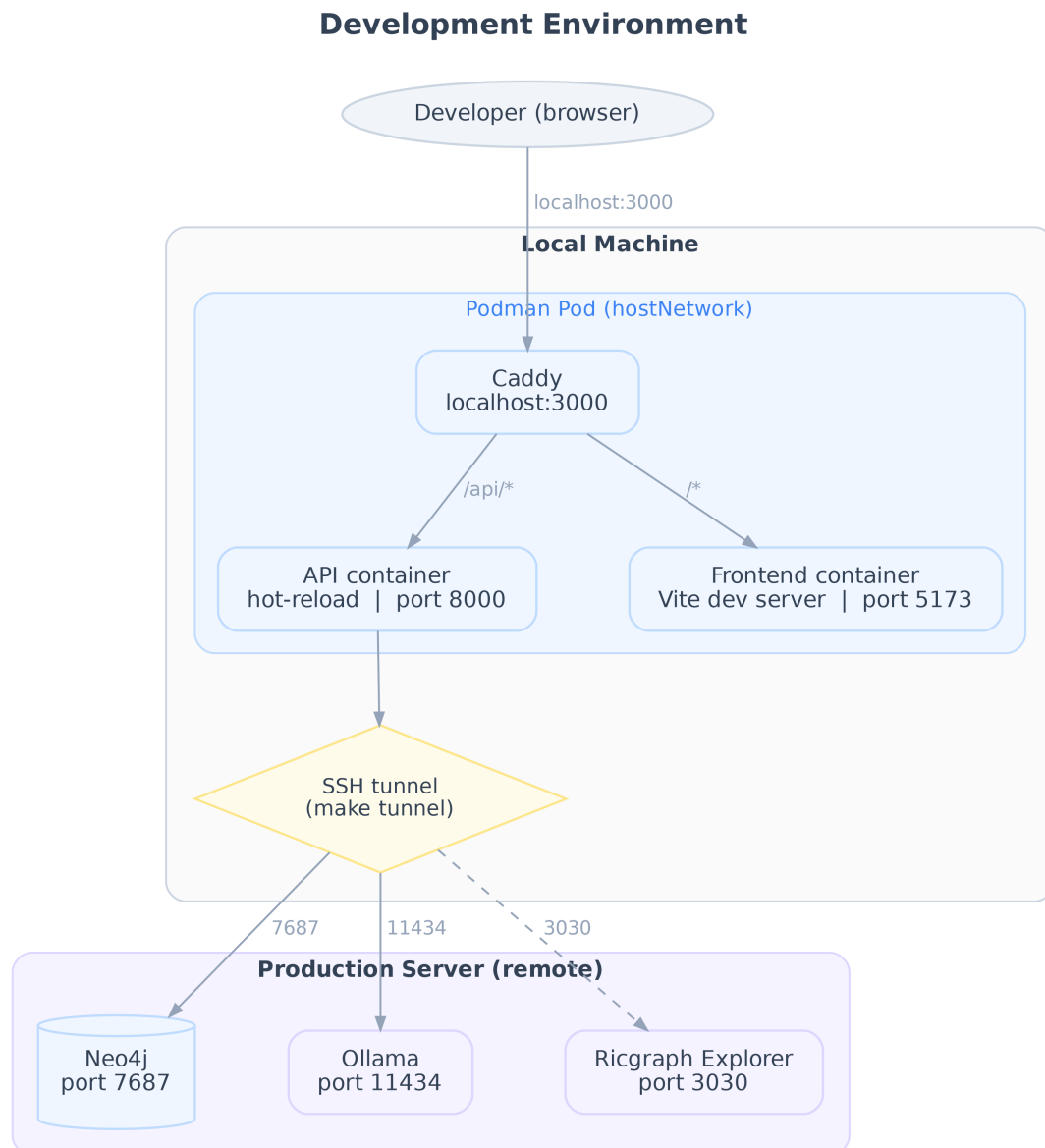


Figure 5.4: Development environment. The API and frontend containers run locally with hot-reload. Neo4j and Ollama are on the production server, reached via SSH tunnel with port forwarding.

5.9.4. Production Environment

The application runs on a single virtual machine provided by SURF Research Cloud. Users do not access the VM directly. Instead, they log in through the SURF Research Cloud portal using their institutional SURFconext credentials and are then given a browser session to the VM. The application itself listens only on localhost within the VM, so no ports are exposed to the internet. This provides network-level isolation without requiring the application to implement its own authentication.

The VM has an Nvidia GPU available for LLM inference. The Ollama container is configured with

access to this GPU so that chat generation with the `command-r:7b` model runs on the GPU while embedding generation with the `snowflake-arctic-embed2` model runs on the CPU. Neo4j stores its data on a persistent volume that survives VM restarts, so the graph does not need to be re-harvested after a reboot.

This deployment model provides strong isolation but comes at the cost of usability, since users must go through the SURF portal before they can use the application. If the legal situation around the aggregated data is resolved in the future, the system can be transitioned to a more publicly accessible deployment. Caddy already supports automatic TLS certificate management and an authentication layer could be added in front of the application.

One of the architectural goals was that the application should be deployable at other institutions without modifying the application code. To achieve this, all connections between services, as well as model selection and API credentials are configured through environment variables rather than hardcoded values. When all containers run on the same machine, they communicate directly over the Podman network using container hostnames. When a service runs on a different machine, the corresponding environment variable can be pointed at the remote address instead.

This means that another institution can adopt the application by adjusting configuration alone. A university with its own Pure installation would set `PURE_URL` to their endpoint. If they prefer a different LLM provider or want to use a cloud-hosted model rather than a local Ollama instance, they can replace `AI_SERVICE_URL`, though this may require an adapter if the provider does not expose an Ollama-compatible API. The data sources that Ricgraph harvests from are similarly configurable through Ricgraph's own settings, so the harvesting step can also be adapted without code changes.

Chapter 6

User interface

The Research AI Assistant enables users to search for a person or research organisation and generate AI-assisted textual outputs based on institutional research data. The application combines structured relationship data with large language model pipelines, allowing users to explore researchers, organisations, publications and collaborations through both direct data views and generated summaries.

6.1. Visual design

Thijmen

The application follows the Utrecht University styleguide,¹ applying the institutional colour palette and typography across all interactive elements, as shown in Figure 6.1. This gives the interface a consistent institutional appearance, in line with the context in which the application is intended to be used.

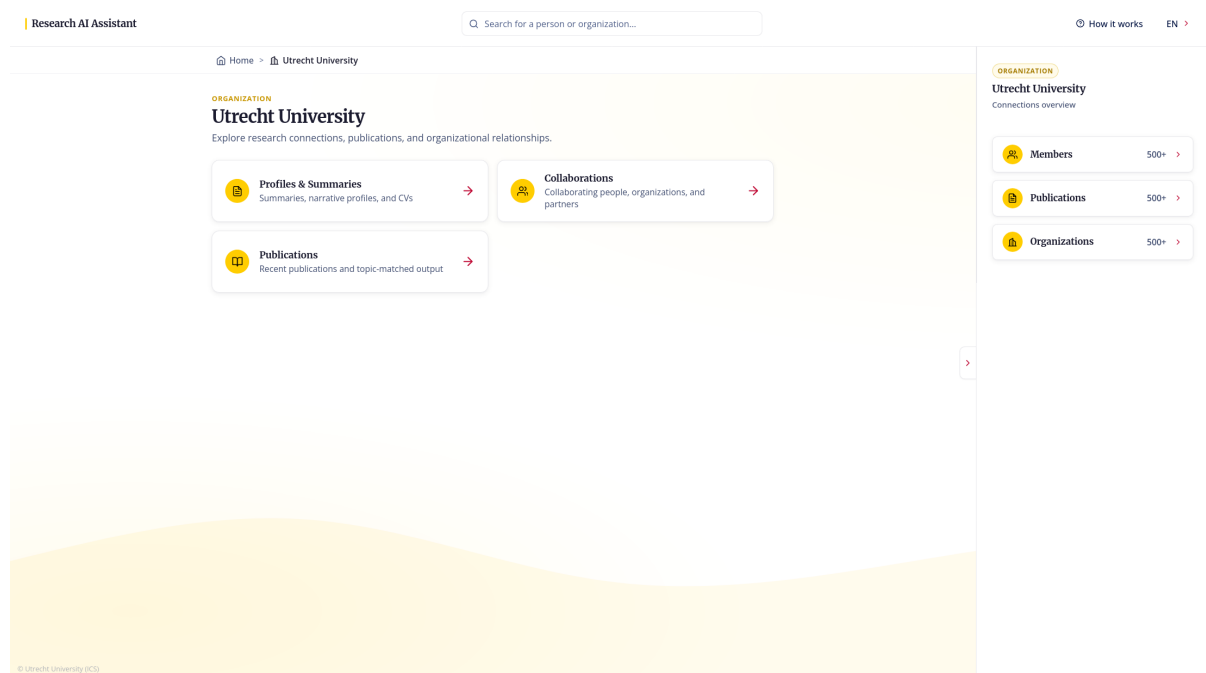


Figure 6.1: The interface with the Utrecht University organisation selected, showing the applied institutional styleguide.

6.2. Searching for entities

Jonathan

The application starts with a single search field on the home page. Users can enter the name of a person or organisation, after which autocomplete suggestions are shown while typing. Both researchers and organisations are searchable. When the user selects a suggestion, the corresponding profile page is opened directly.

¹<https://www.uu.nl/organisatie/huisstijl>

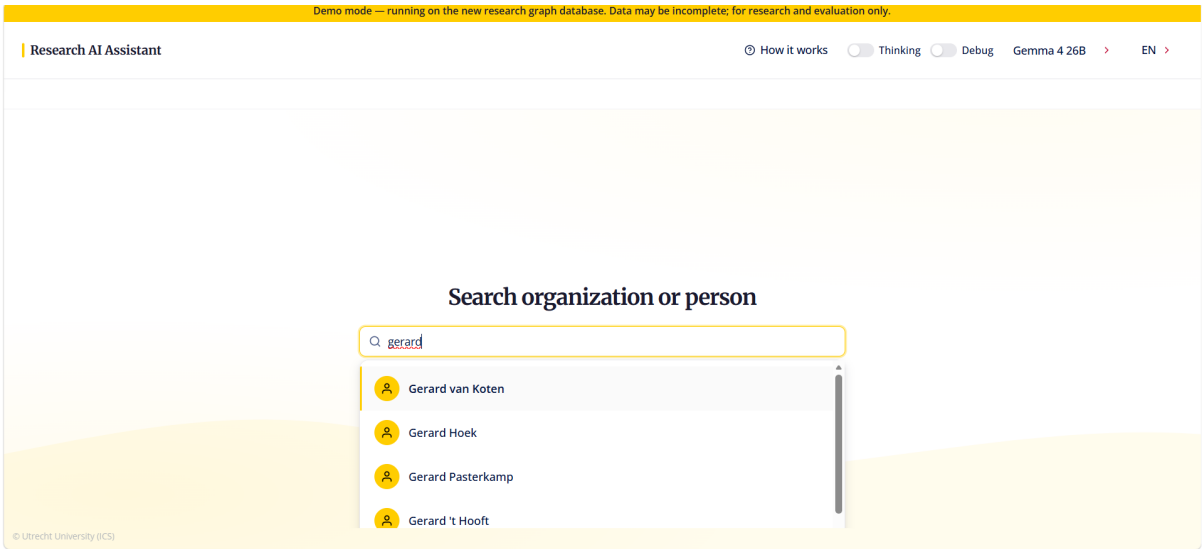


Figure 6.2: Home page search field showing autocomplete suggestions while typing.

The search field remains available in the header on all profile pages, as shown in Figure 6.3. This allows users to move from one entity to another without returning to the home page, supporting a continuous exploration workflow.

6.3. Profile page

Jonathan

After selecting a person or organisation, the application displays a profile page consisting of two main areas. The central column contains the available prompt categories and displays the AI-generated output. The right side of the page contains the connections panel, which presents structured information from the dataset, such as related people, publications and organisations.

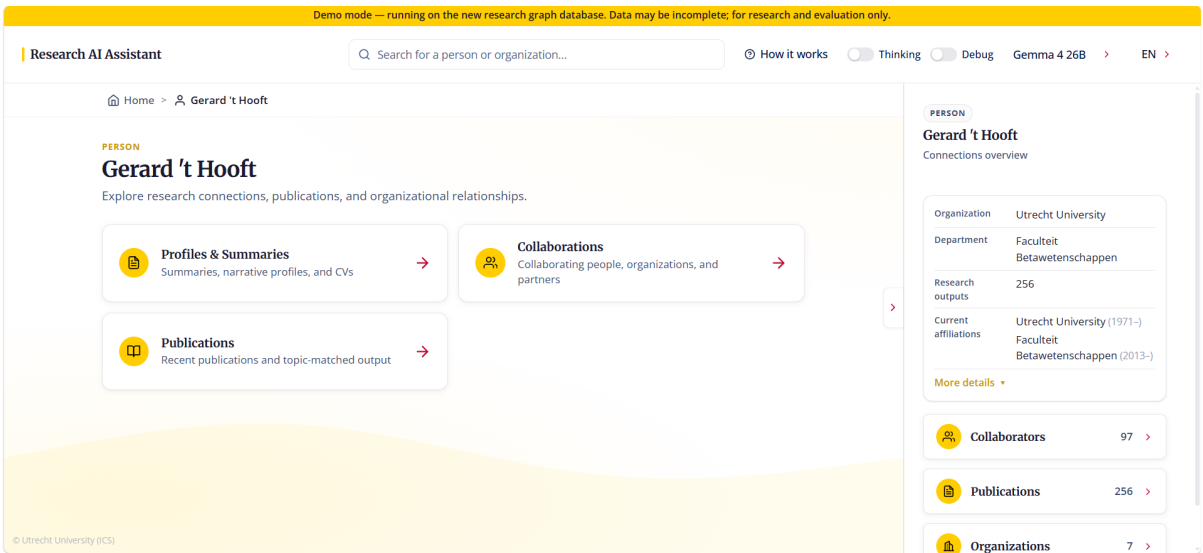


Figure 6.3: Profile page on a desktop viewport, showing the header search field, the prompt categories, and the connections panel with organisation, department, and collaboration counts.

On smaller screens, the layout adapts automatically. The connections panel becomes accessible through a collapsible mechanism, while the prompt categories remain stacked in a single column. This ensures that the same functionality remains available on mobile devices, while only the visual

arrangement changes.

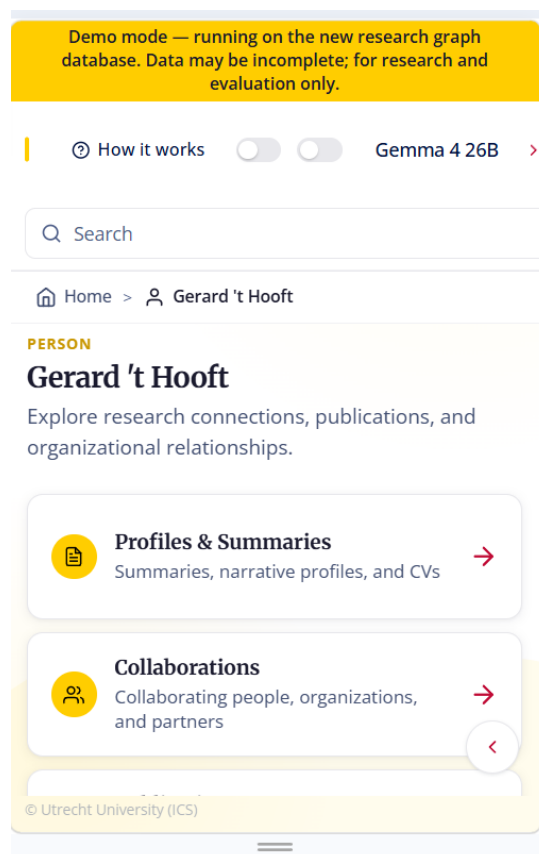


Figure 6.4: Profile page on a mobile viewport, with the connections panel collapsed behind a toggle on the right edge.

6.4. Connections panel

Jonathan

The connections panel provides a structured overview of the relations recorded in the dataset through Ricgraph. Unlike the AI-generated responses, the data in this panel is shown directly from the underlying dataset. This makes the panel useful as a transparent reference point alongside the generated text, as illustrated in Figure 6.3.

The panel is divided into three sections. The people section lists employees, members or collaborators connected to the selected entity and can be sorted alphabetically. The publications section shows publication records and supports filtering by year, sorting by year or title, and access to DOI-related actions. The organisations section lists institutional relationships connected to the selected entity.

Large result sets are loaded incrementally through a “Load more” button. For publications, the year filter also affects several AI prompts. When a user selects a year range in the publications section, that range is automatically reflected in prompts such as summaries, narrative profiles, NWO-style CVs and recent publication overviews.

6.5. AI prompt categories

Jonathan

The application offers AI-generated output through prompt categories. Each category opens a modal window containing the relevant prompts, as illustrated in Figure 6.5. The available prompts

depend on the type of entity selected, since some prompts are only meaningful for persons while others are designed for organisations.

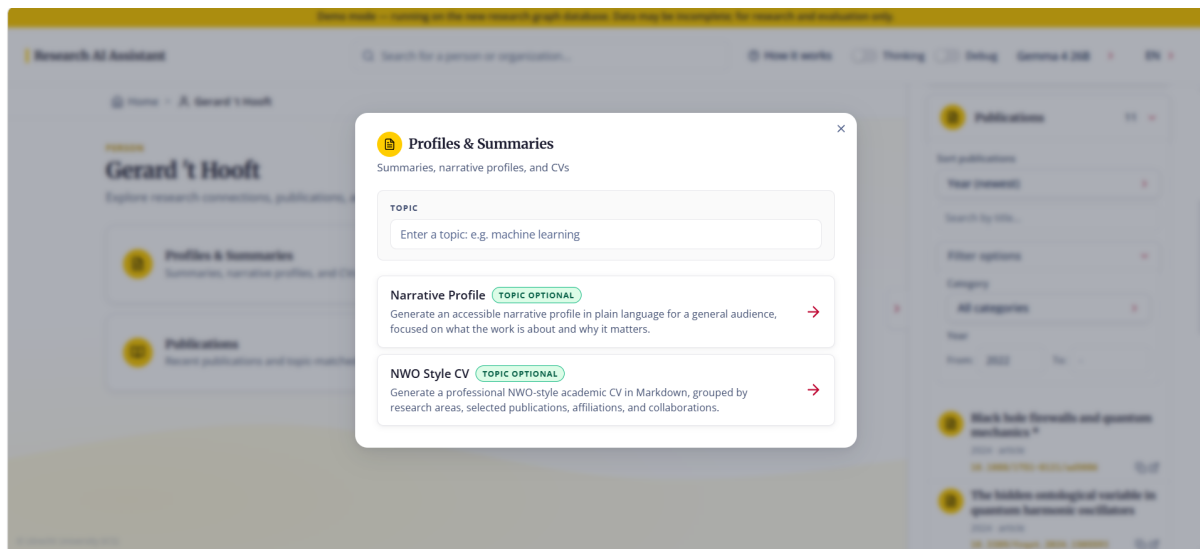


Figure 6.5: The Profiles & Summaries modal for a person, showing the Narrative Profile and NWO Style CV prompts together with the optional topic input field.

6.5.1. Profiles and summaries

The profiles and summaries category contains prompts that generate profile texts and CV-style outputs. For a person, the available prompts are *Narrative Profile* and *NWO Style CV*. The narrative profile generates an accessible profile in ordinary language, intended for a broad audience. The NWO-style CV produces a more academic output, structured around research lines, publications and collaborations.

For an organisation, only the narrative profile is available. In both cases, the user may optionally provide a topic to influence the emphasis of the generated text.

6.5.2. Collaborations

The collaborations category focuses on relationships between researchers and organisations. For a person, the available prompts are *Top Collaborating Organizations*, *Top External Collaboration Partners* and *Suggest External Partners*. These prompts identify frequent institutional collaborators, frequent external co-authors and possible external researchers with similar research topics.

For an organisation, the corresponding prompts are *Related Organizations*, *Top External Collaborating People* and *Suggest External Partners*. These prompts are based on collaboration patterns in the dataset and do not use the optional topic field.

6.5.3. Publications

The publications category focuses on research output. For both persons and organisations, the available prompts are *Recent Publications*, *All Output on Subject* and *Top 5 Research Topics*. The recent publications prompt provides an overview of the most recent publications, optionally prioritising publications related to a topic entered by the user. The all output on subject prompt lists publications that match a required topic. The top five research topics prompt identifies recurring themes based on publication titles and abstracts.

For organisations, an additional prompt is available: *Find Experts on Topic*. This prompt identifies

members of the organisation whose publication output is most relevant to a required topic.

6.6. Optional topic input

Jonathan

Below the prompt categories, the interface provides a topic input field, visible in Figure 6.5. Users can enter a topic such as climate, education or heart disease. This topic affects some prompts but not all of them. For the summary and narrative profile prompts, the topic influences the focus and emphasis of the generated text. For recent publications, it prioritises publications related to the topic. For the NWO-style CV and the collaboration-related top prompts, the topic field has no effect.

6.7. Generated output, sources and navigation

Jonathan

After a prompt is selected, the generated text appears in the central output area. Multiple outputs can be generated after one another, with each response shown as a separate block. Each block has its own delete button, and the user can stop an active generation using the stop button.

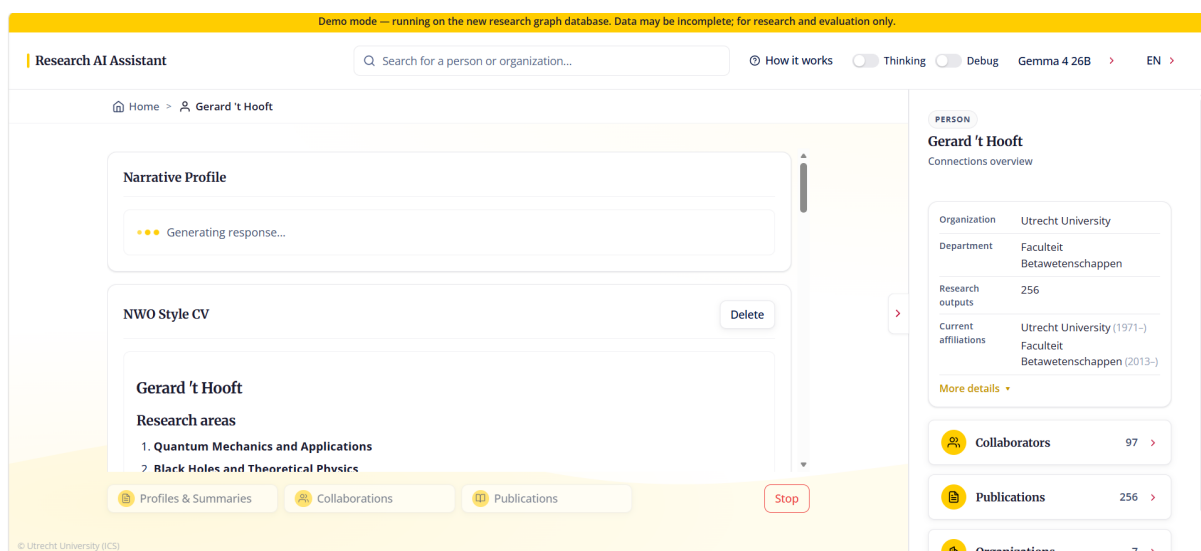


Figure 6.6: Central output area with a narrative profile still generating and a completed NWO style CV block with its own delete option; the prompt categories and a stop button remain accessible at the bottom.

Once a response has been generated, a source list is shown below the text. Numbered citations in the generated response refer to publication records in this list. Users can click these sources to inspect more detailed publication information. Names of people and organisations mentioned in generated text are often rendered as internal links, allowing the user to navigate directly to another profile page.

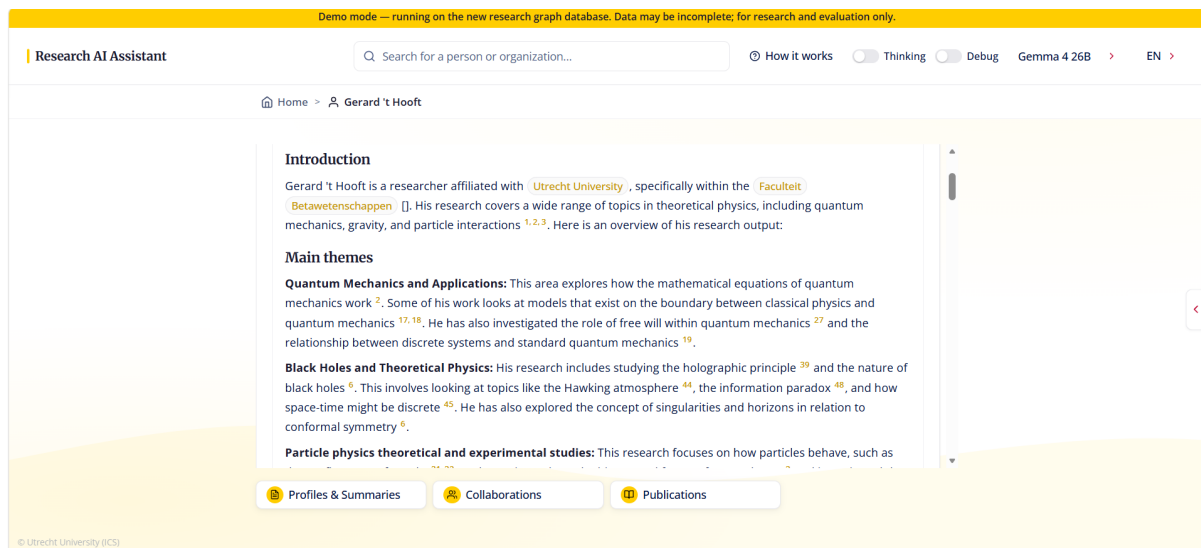


Figure 6.7: Generated narrative profile with numbered citations referring to publication records, and internal links to the mentioned organisation and department.

When the available data is insufficient, the model is expected to state this explicitly instead of speculating. This supports transparency and helps users distinguish between information grounded in the dataset and information that cannot be reliably inferred.

6.8. User documentation

Thijmen

The application includes a built-in documentation page that provides contextual help without requiring external materials. It explains how the search works, how to interpret the connections panel, how the AI prompts behave, how the numbered citations relate to their sources, and how generated text should be read and evaluated critically. This gives users immediate access to support from within the application itself.

6.9. Language support

Jonathan

The application is available in both English and Dutch. Users can switch languages using the language button in the header. The language setting applies to the interface as well as to the built-in documentation page. This bilingual design reflects the institutional context in which the application is intended to be used.

Chapter 7

Test approach

Author: Timon | Reviewer: Lukas

This section describes our testing approach as it was executed and reports on the results obtained. The structure follows the two pillars of our testing approach, functional and non-functional testing, before going over the results and key changes.

7.1. Quality aspects

The important quality aspects of our project were noted in a previously written piece, our test plan, see section ?? We prioritise maintainability, performance efficiency, reliability, portability and usability. For each of these quality aspects, a testing decision was made on how to incorporate them. For example, for maintainability, we made sure our test structure was designed in such a way to easily modify or add any tests. Furthermore, tests were made to ensure the performance efficiency is up to par with what our client expected. Additionally, in regards to portability, we created integration and system tests that test the system as a whole, which is important when the database or LLM model is swapped out. Our testing strategy is based on these important quality aspects.

7.2. Functional testing

In projects like ours, having well-defined test strategies across multiple abstraction levels is necessary. Our testing strategies, focus on the backend API and thus tests are grouped within subdirectories to reflect their abstraction levels (e.g. unit, integration and system). Within those directories are the actual files that contain the tests, which are also divided neatly into classes and functions.

The most basic tests are unit tests. Our unit test strategy makes sure that the smallest parts of our backend API are tested as follows: we aim to create useful unit tests for all our various utility functions, schemas/classes, endpoints, and helper scripts. These unit tests involve a lot of mocking and therefore do not require real data or server access. We also try to keep them as simple as possible while being thorough and testing edge cases.

Our integration tests focus on validating that our API correctly handles incoming requests and routes requests appropriately. This obviously means that the backend code must be running locally, such that requests can be sent and received. Our strategy is to make sure that the most basic aspects of our API calls function properly, which means that we also require connections to our database and the LLM. We do not validate any of the returned data, but rather check if requests are accepted or denied when expected, whether responses include any data, and that the responses are in their proper formats.

The most high-level tests which we use, called system tests, have the purpose of validating the entire API pipeline. Our strategy is to make API requests to the actual production server and validate that we get an appropriate response which is only possible if all the required connections are established and functioning correctly.

7.3. Non-functional testing

Unfortunately, not all quality aspects can be addressed by functional tests, which is why we used several other quality assurance instruments. Listed here are some of our most important QA instruments.

- Documentation: documentation clearly defines what the code is expected to do, and thus helps us stay on track and more easily develop tests.
- Proper pull requests: by using pull requests, which come with built-in checklists and AI code review, that can only be merged if at least two collaborators approve the code, we prevent a lot of bad code from reaching our code base.
- Team agreements: we have internally discussed and documented how we wish to work together, this also includes some agreements concerning how we write code. Code comments, AI usage and the importance of clear structure are among the topics discussed. This will help ensuring proper quality throughout this project.
- Quality inspections: over the course of this project, there are multiple code quality inspections by teaching assistants, this helps us get an outside look into our code base which might catch mistakes that we overlooked.

Most of the non-functional requirements, if they are testable, are hard to quantify and are therefore hard to write useful tests for. We have solved this by having at least two persons review new code to ensure quality aspects, and we also employ user testing. This means letting potential future users use the system for their various purposes, collecting their feedback, and using those issues as both checklists for future versions and immediate required changes. To make sure we receive useful feedback, we will provide a questionnaire that users will need to walk through. Users will also be encouraged to test their specific use cases, since we do not have the same expertise as our users, resulting in valuable and accurate feedback regarding the accuracy of the LLM.

7.4. Functional test results

The functional testing strategy described in Section 7.2. was executed in the intended layers. Table 7.1 summarizes the resulting test suite at the end of the project.

Table 7.1: Functional test results across the three testing layers.

Test layer	Number of tests
Unit tests	874
Integration tests	17
System tests	12
Total	903

Overall code coverage: **96%**.

The distribution of tests across the three layers follows our intended test strategy. This consists of a wide variety of unit tests, a smaller group of integration tests followed by a small amount of integration tests. The significantly larger number of unit tests stems from their nature of being inexpensive and maintainable. Due to our definition of done, unit tests had to be made every time new code was written. This led to the number of unit tests increasing over time, while the number of integration and system tests remained the same since the start.

The 96% code coverage is solely based on the backend of the codebase. Coverage of this mag-

nitude indicates that most of the backend code is covered by at least one test. This does not guarantee correctness, but rather helps preventing mistakes in the future.

7.5. Non-functional test results

Results for non-functional testing are hard to quantify however, there are a couple of QA instrument results worth mentioning. Over the course of this project a total of three quality inspections were conducted by the teaching assistant. With each inspection the codebase quality improved greatly, resulting in all green sections on the latest inspection.

Our most reliable source of non-functional testing was conducted via user testing. Test users were provided by the client, each in their own expertise. Multiple rounds of user testing were conducted, giving valuable feedback via a questionnaire that our team overlooked or simply could not verify. The accuracy of the LLM was the main focus point in these test settings. The user tests mostly helped us determine the accuracy level of our LLM, followed by user interface bugs and feature requests.

7.6. Retained elements

The test file structure with its abstraction levels (e.g. unit, integration and system) remained the same throughout the whole project. A clear distinction between the different levels of testing was a very efficient choice in our project and made it easy to maintain our testing code. The various QA instruments used at the start of this project remained mostly the same. Documentation had to be written for all new code, pull requests had to go through a checklist, and two reviewer approvals in order to be accepted, and the three quality inspections provided us with useful feedback to improve our codebase.

7.7. Revised elements

The QA instrument that did change was the team agreements, in particular the agreements on the pull request process. The requirement of two reviewers on every pull request did not change, however the way we made the pull requests and asked for reviewers did. At the start of this project this was defined very loosely, it resulted in pull requests opening up late in the sprints, having to wait a long time on reviews and thus a high branch lifetime and difficult merges. So, the agreements had to be adjusted to improve the situation. With the new agreements, reviewing pull requests was our number one priority in the sprint, before working on other tasks we decided that everyone had to check if a pull request was open and needed reviews. At the same time, the person who opened the pull request had to ask for reviews either physically, when we were in the office, or online. This change significantly shortened our branch lifetime and prevented large merge issues.

The initial plan for user testing consisted of the user having to achieve predefined goals to verify whether our program is usable in regular use cases. In stead of having predefined goals, we made a questionnaire that guides the user through the website. This decision was made based on the fact that the users who tested our program worked within the Utrecht University and thus could more easily verify the LLM output in their own expertise. This resulted in more valuable feedback on the accuracy of the LLM output. At the end of the session we always asked if the user had any remarks on the UI and we asked what feature the user would implement if there were no constraints.

Chapter 8

Product Handover and Potential Future Development

Author: Sybren | Reviewer: Jelmer

The project concluded with the delivery of the final version of our product to the client. This section discusses how the product was prepared for handover, how certain limitations influenced decisions about the final product and how development could continue in the future.

8.1. Handover Preparation

To ensure that the client can maintain and extend the system independently several measures were taken during the final stages of the project. These measures target the two non-functional requirements most relevant to a handover, namely portability and maintainability (see Section 3).

The deployment process has been consolidated into a single Makefile. As described in Section 5.9.1., deploying a new version requires only an SSH connection and two commands. This lowers the barrier for the RDMS department to update the application without needing detailed knowledge of the underlying container infrastructure. Service logs are accessible through standard systemd tooling and do not require familiarity with custom monitoring solutions.

Portability was addressed by externalising all service connections and credentials into environment variables rather than hardcoding them (see Section ??). Another institution adopting the system can point it at their own Pure installation or LLM provider by adjusting configuration alone without modifying application code. The use of Podman with Quadlet (see Section 4.4.) supports this further by avoiding dependencies on proprietary orchestration tooling.

The codebase includes inline documentation and a structured file layout (see Section 5.6.) intended to make the purpose of each module identifiable without external guidance. The user-facing documentation built into the application (see Section 6.8.) allows end users to understand the system without developer involvement.

Certain aspects of the handover remain limited. No formal knowledge transfer sessions were conducted with the client's development team and the deployment process is not automated through a continuous integration pipeline. These are areas where future work could reduce the operational burden.

8.2. Limitations

The context in which this project was undertaken introduced various constraints that had to be worked around while still ensuring a satisfactory outcome. Examples of these limitations include the allotted time, the overhead incurred by course activities and the size of the team. Other important factors specific to our team were limited experience and limited domain knowledge. This section goes over the two most important decisions regarding the final system that were influenced by these limitations. While many other decisions were also shaped by these constraints, the two discussed below impact the functionality of the final product instead of merely slowing down progress or causing inefficiencies.

8.2.1. Security

One critical aspect of systems used in institutional settings that is notably absent from our application is security. The reasons for omitting this important component and placing it within the *Won't have* category (see Section 3) have to do with the previously mentioned limitations. Ideally, the system would implement authentication via SURFconext after which only authorized users could access and interact with the data. However, adding this functionality to our system was outside the scope of this project, as our team neither possessed the required expertise nor had sufficient time to implement and validate proper security systems. As a result, the decision was made to develop the application without explicit security systems but to keep the application private and restrict the usage of our system to select individuals.

8.2.2. Open Calls

Another feature that was out of scope for the final product was the ability to view and query the LLM about open calls¹. Such a feature would address a typical use case for our stakeholders, as researchers could find funding opportunities for future projects.

We decided not to include this feature because gathering and organizing the required data while also providing ways to interact with it would not allow us to reuse any components of our software. Instead, it would require entirely new systems to be made. For example, open calls cannot be classified among research entities such as persons or organizations and would therefore require new systems to search for them, include them in various RAG pipelines and show them in the UI. Another issue with this feature is that, considering the financial nature of the funding calls, the LLM output should be extremely precise and reliable, which as discussed previously is hard to guarantee. Thus, the allotted time constrained our ability to fully implement this feature.

8.3. Future Development

Should development of this project continue in the future there are several categories of improvement that could be pursued. The items below are roughly ordered by how directly they build on the existing architecture.

The two features discussed in Section 8.2., authentication via SURFconext and support for open calls, remain the most impactful additions. For authentication the current deployment already provides network-level isolation through the SURF Research Cloud portal (see Section 5.9.4.). Adding application-level authentication would allow the system to be moved to a publicly accessible deployment. Caddy is already prepared to support this through automatic TLS certificate management (see Section 4.4.). Open calls would require more substantial changes to the data model and pipeline infrastructure as discussed above.

Closer to the existing implementation the search functionality could be improved with filters for publication year or organisation type. The database could also be extended to include additional data types such as activities and prizes. This would give the LLM more context to draw on when generating outputs. Both changes would operate within the current architecture and would not require modifications to the pipeline framework (see Section 5.4.).

Further from the current system the user experience could be enhanced by visualising entity relationships as an interactive knowledge graph rather than only presenting them as lists in the connections panel. An admin panel would allow authorised users to manage permissions and monitor usage. This would become particularly relevant if the application is opened to a broader audience after authentication is implemented.

¹In this case, open calls refer to calls put forward by organizations such as NWO for researchers to acquire funding for certain projects. See <https://www.nwo.nl/en/calls>

Finally the deployment process could be improved by introducing a continuous integration pipeline that runs the test suite on each commit and deploys validated builds to the server. This would reduce the manual effort currently required (see Section 5.9.1.) and lower the risk of deploying untested changes.

To conclude, the final product implements what was achievable within the scope and constraints of this project while leaving room for future additions. Making the code maintainable and understandable was a priority in this project, meaning future development should be manageable and should not require major changes to existing code.

Chapter 9

Process

The Research AI project was carried out using a Scrum-based approach. This approach was selected because the project contained technical uncertainty, multiple interdependent components, and requirements that had to be validated during development. The product combined frontend development, backend services, AI functionality, retrieval-augmented generation, data integrations, infrastructure, testing, documentation, and user validation. A fixed sequential approach would have required many decisions to be made before enough information was available. Scrum made it possible to work iteratively while still maintaining planning, review, and control. This chapter focuses on the organisation of the development process.

9.1. Scrum Framework

Anouar

Scrum was used as the main framework for organising the project. The team worked in sprints of approximately two weeks. Each sprint followed a recurring structure consisting of sprint planning, development work, intermediate coordination, a sprint review with the client, and a retrospective. This structure created fixed moments for planning, execution, validation, and process evaluation.

The Scrum approach was applied in a practical way. The team used the framework to divide work into manageable tasks, monitor progress during the sprint, collect feedback from the client, and adjust the planning when new information became available. Because the project involved several technical components, Scrum also helped to make dependencies between tasks visible.

The main Scrum artefacts used during the project were the product backlog and the sprint backlog. The product backlog contained user stories, technical tasks, documentation tasks, and process-related action points. The sprint backlog contained the tasks selected for the current sprint. These artefacts were updated throughout the project when new insights, technical findings, or stakeholder feedback affected the planning.

9.2. Sprint Backlog and Task Management

Anouar

The team used a sprint backlog to organise and monitor the work for each sprint. At the beginning of a sprint, the team reviewed the work that still had to be done and selected the tasks that were relevant for the upcoming sprint. These tasks included user stories, technical implementation tasks, documentation tasks, testing tasks, and process-related action points.

The sprint backlog was updated during the project when new information became available. Feedback from the client, technical findings, integration issues, or changes in priority could lead to tasks being added, adjusted, or moved to a later sprint. In this way, the sprint backlog functioned as the main practical planning tool for the team.

The backlog was not split into a strict product backlog and sprint backlog. Instead, the team mainly worked with the sprint backlog as the central overview for the current planning. At the start of each sprint, this overview was reviewed and updated based on the current state of the project. This approach suited the project because the team often had to respond to new technical insights and stakeholder feedback.

The sprint backlog also helped the team distinguish between different types of work. Some tasks were directly related to visible product functionality, such as frontend components, backend endpoints, search functionality, or AI-generated output. Other tasks supported the technical foundation of the system, such as testing, bug fixing, documentation, performance improvements, and

integration work. Although these supporting tasks were less visible to the client, they were necessary for the reliability and maintainability of the product.

As the project progressed, task descriptions became more specific. Instead of only describing broad goals, the team increasingly defined tasks with a clear expected outcome, responsible team member, and relevant dependencies. This made the sprint backlog more useful as a planning and coordination tool.

9.3. Sprint Planning

Anouar

Sprint planning was held at the beginning of each sprint. During this meeting, the team determined which tasks would be addressed during the upcoming sprint and how the work would be divided. The selection of tasks was based on the current state of the project, client feedback, technical findings, dependencies between components, and the available capacity of the team.

The team used a Scrum board to structure and monitor the sprint work. This board consisted of several swimlanes, in which tasks were grouped according to their topic, component, or area of responsibility. Examples of such areas included frontend development, backend development, AI functionality, data integration, infrastructure, testing, and documentation. The use of swimlanes made it easier to maintain an overview of parallel workstreams and to identify where dependencies between tasks existed.

Within the Scrum board, larger tasks were divided into smaller subtasks where possible. Instead of assigning a small number of broad tasks, the team aimed to define tasks at a more detailed level. This made the work more manageable and improved progress tracking during the sprint. Smaller tasks also made it easier to assign responsibility, review progress, identify blockers, and complete work incrementally.

Tasks were divided based on the type of work, the required expertise, and the relationship between components. Some team members worked mainly on frontend functionality, while others focused on backend development, AI functionality, data integration, infrastructure, testing, or documentation. This division was not fixed throughout the project, because several tasks required collaboration between multiple areas.

To keep responsibilities clear, tasks were assigned to specific team members or smaller subgroups. This made it easier to monitor progress and determine who was responsible for completing or coordinating a task. When a task affected multiple parts of the system, the relevant team members coordinated through standups, office days, or separate technical discussions.

Sprint planning was also used to balance different types of work. In the early sprints, more attention was given to building the technical foundation and implementing core functionality. In later sprints, the focus shifted towards integration, testing, performance, error handling, prompt improvement, documentation, and processing user feedback.

9.4. Daily Standups and Coordination

Anouar

Sprint planning was held at the beginning of each sprint. During this meeting, the team determined which tasks would be addressed during the upcoming sprint and how the work would be divided. The selection of tasks was based on the current state of the project, client feedback, technical findings, dependencies between components, and the available capacity of the team.

The team used a Scrum board to structure and monitor the sprint work. This board consisted of several swimlanes, in which tasks were grouped according to their topic, component, or area of responsibility. Examples of such areas included frontend development, backend development, AI functionality, data integration, infrastructure, testing, and documentation. The use of swimlanes

made it easier to maintain an overview of parallel workstreams and to identify where dependencies between tasks existed.

Within the Scrum board, larger tasks were divided into smaller subtasks where possible. Instead of assigning a small number of broad tasks, the team aimed to define tasks at a more detailed level. This made the work more manageable and improved progress tracking during the sprint. Smaller tasks also made it easier to assign responsibility, review progress, identify blockers, and complete work incrementally.

Tasks were divided based on the type of work, the required expertise, and the relationship between components. Some team members worked mainly on frontend functionality, while others focused on backend development, AI functionality, data integration, infrastructure, testing, or documentation. This division was not fixed throughout the project, because several tasks required collaboration between multiple areas.

To keep responsibilities clear, tasks were assigned to specific team members or smaller subgroups. This made it easier to monitor progress and determine who was responsible for completing or coordinating a task. When a task affected multiple parts of the system, the relevant team members coordinated through standups, office days, or separate technical discussions.

Sprint planning was also used to balance different types of work. In the early sprints, more attention was given to building the technical foundation and implementing core functionality. In later sprints, the focus shifted towards integration, testing, performance, error handling, prompt improvement, documentation, and processing user feedback.

9.5. Sprint Reviews

Anouar

At the end of each sprint, a sprint review was held with the client. During this meeting, the team presented completed work, demonstrated implemented functionality, and discussed relevant findings. The review was used to validate whether the delivered work matched the expectations of the client.

The team used a structured presentation format for the sprint reviews. Team members added slides about the work they had completed, the functionality they had implemented, and any relevant technical or organisational points. The chairperson guided the meeting, while contributors presented the work they had been responsible for.

The sprint review also functioned as a moment for collecting feedback. Feedback from the client could lead to new backlog items, changes in priority, or further investigation. Not all feedback automatically resulted in a scope change. Feedback was first assessed in relation to the project goals, technical feasibility, available time, and existing requirements.

This approach helped to keep the sprint review focused. The meeting was not only used to show progress, but also to determine whether the project direction still matched the client's expectations.

After each sprint review, the team held a retrospective. The retrospective was used to evaluate the development process rather than the product itself. During this meeting, the team discussed what went well, what caused problems, and which improvements were needed for the next sprint.

The retrospective focused on recurring process topics such as task clarity, communication, planning, pull request timing, Git workflow, code review responsibility, and meeting attendance. These topics were relevant because process issues could directly affect the quality and integration of the product.

The outcome of a retrospective was translated into concrete action points. For example, when pull requests were opened too late, the team agreed to open them earlier in the sprint. When task

descriptions were too broad, the team made them more specific. When Git-related issues occurred, the team made clearer agreements about branches, reviews, and integration.

Holding the retrospective directly after the sprint review made it possible to include client feedback in the next planning cycle. This created a clear transition from evaluation to improvement and then to planning for the next sprint.

9.6. Code Review and Integration

Anouar

Code review was part of the development workflow. Pull requests were used to review changes before they were merged. This allowed team members to check whether code was understandable, maintainable, and aligned with the existing implementation.

Integration was an important part of the project because many components depended on each other. Frontend functionality had to connect to backend endpoints, backend services had to retrieve and process data, and AI-generated output depended on the quality of the retrieval process. For this reason, code changes could not be assessed only in isolation.

When pull requests were opened late in a sprint, review and integration work became concentrated near the sprint deadline. To reduce this issue, the team introduced earlier pull request moments and gave open pull requests more attention during the sprint. This made it possible to identify integration problems earlier.

A dedicated pull request channel was also used to make review requests more visible. This supported the review process and reduced the chance that pull requests remained unnoticed.

Scrum proved to be a well-suited framework for this project. The core justification lies in the level of uncertainty the project involved. Many decisions could not be made responsibly at the start, because the necessary insight only emerged during development through experimentation and client feedback. Scrum created the space to make those decisions at the right moment, rather than being forced to commit to them before enough information was available.

That said, the iterative structure alone did not guarantee smooth execution. The most persistent challenge throughout the project was the late opening of pull requests, which repeatedly concentrated integration work near sprint deadlines. This pattern recurred across multiple sprints despite being identified early as a risk, which suggests that recognising a process problem and actually solving it are two different things. The introduction of a dedicated pull request channel and earlier PR deadlines did lead to clear improvement by sprint 7, but the time it took to get there reflects a broader lesson: process agreements only work when the team actively holds itself to them.

A second recurring tension was between feature development and technical quality. Around sprint 3, the client explicitly prioritised code quality and documentation over new functionality. This aligned with signals the team was already observing internally. The decision to take that feedback seriously and shift focus, rather than continuing to push new features, was the right call. Investing in quality at that stage paid off in later sprints, where the improved foundation made it significantly easier to build on and extend existing work.

Stakeholder management also required careful handling. The client side was not always aligned: in sprint 3, conflicting input from different stakeholders meant the team had to seek direct clarification rather than acting on unclear instructions. Learning to distinguish between feedback that should be acted on and feedback that should be questioned turned out to be an important part of working effectively with the client.

In retrospect, the project shows that choosing the right method is only part of the story. Scrum provided the structure, but the quality of the outcome depended on how consistently the team

engaged with that structure; in planning, in review, and in the willingness to treat agreed improvements as commitments rather than suggestions.

9.7. Process of building Pipeline code

Ivan

The team went through a number of iterations before settling on the current pipeline strategy.

9.7.1. Challenges

The main challenge with the pipelines is that an LLM can only process a limited number of tokens, and for a local model this context window is quite small. Much of the code is therefore dedicated to reducing a sometimes enormous list of publications to just a few, enough for the model to produce a well-considered and accurate answer. The central question here was how to decide which documents are the most relevant to provide. This is made more difficult by the fact that different situations (such as a higher publication count, or whether or not a topic is given) require different approaches.

Performance was another challenge. One requirement from the client was that each request should take at most around 30 seconds. The team initially dismissed this concern, as the system was already considerably faster than that. However, the team had not accounted for the fact that it was working with only a very small database at the time. When the team began working with a full database, streaming became much slower, especially for large organisations. Several measures were therefore taken to improve performance.

9.7.2. Earlier approaches

The current structure did not emerge immediately. Broadly, the code went through three major phases (see Figure 9.1).

Phase 1: Separate modules. At the start of the project, each pipeline had its own code, with nothing shared between them. This became unmanageable as more pipelines were added.

Phase 2: Monolithic service. All logic was placed in one large service class, and each product had a single method that was called. A few shared modules existed as plain functions within the same large file, and the HTTP layer held a table mapping prompt names to their respective methods.

Phase 3: Object-oriented design. This is the current approach. The team divided each pipeline into small pieces and then identified which pieces could be reused by other pipelines. A detailed specification document was written to outline all of the required modules, after which the pipeline classes, reusable building blocks, and registry were built. The following weeks were spent optimizing this approach.

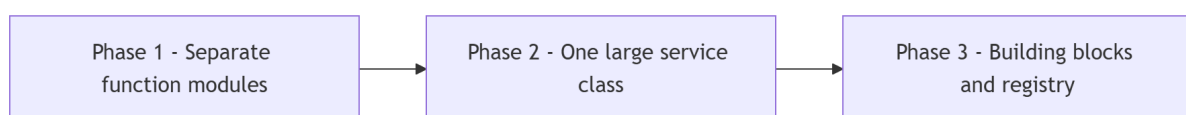


Figure 9.1: Timeline of pipeline approaches

Chapter 10

Attachments

Jonathan

10.1. Sprint summaries

10.1.1. Sprint 1 Summary

10.2. Project Plan

Project Plan — Research AI

Triple One Intelligence:

Jelmer Andre de la Porte 6578772

Ivan Balkenende 1196340

Jonathan Bosker 2046636

Anouar Boubkari 1747479

Lukas van Dee 1048708

Thijmen Ligter 4687582

Sybren van der Meijden 2077523

Onno Meppelink 0989711

Timon Zoutendijk 4611659

Monday the 22 of June 2026

Contents

1 Project Context (Jonathan)	4
1.1 Project Objective	4
1.2 Client and Organizational Context	4
1.3 Problem Statement	4
1.4 Role of Pure, Ricgraph, and OpenAlex	4
2 Stakeholders and Their Interests (Jonathan)	5
2.1 Utrecht University (ITS) - client	5
2.2 Research Staff	5
2.3 Researchers	5
2.4 Universities (Future Adopters)	5
3 MoSCoW Prioritization and Key Epics (Lukas/Onno)	6
3.1 Must Haves	6
3.2 Should Haves	6
3.3 Could Haves	7
3.4 Won't Haves	7
3.5 Key Epics	8
3.5.1 User authentication	8
3.5.2 Usability	8
3.5.3 AI interaction	8
3.5.4 Product portability & modular integrations	8
3.5.5 Search functionality	9
3.5.6 Documentation	9
3.5.7 Phone (mobile) adaptation	9
3.5.8 Quality assurance	10
4 MVP Description (Lukas)	10
4.1 Overview	10
4.2 What does the MVP Deliver?	10
4.3 Technical Foundation	11
4.4 What the MVP Does Not Include	11
4.5 Success Criteria	12
5 Scrum implementation (Ivan)	12
6 Risks (Timon)	14
6.1 Stakeholder Risks	14
6.2 Communication Risks	14
6.3 Data Risks	14
6.4 Knowledge Risks	15
6.5 Technical Risks	15
7 Global Planning (Onno)	16

8 Technical and Domain Knowledge (Anouar) 18

8.1 Domain knowledge 18

8.2 Technical knowledge 18

9 AI Statement (Jonathan) 19

1 Project Context (Jonathan)

1.1 Project Objective

The objective is to deliver a Research AI Portal: a portable and open-source web application that provides AI-assisted access to institutional research information in a transparent and user-friendly manner. The portal enables authenticated users to query and explore research information at the level of persons, organizations, and collaborations through predefined, high-quality prompts or by custom user prompts.

1.2 Client and Organizational Context

This project is carried out for Utrecht University’s Information and Technology Services (ITS): Research and Data Management Services (RDMS). RDMS manages and develops research-supporting applications and infrastructures used within Utrecht University and, in some cases, by other institutions. The system to be delivered will operate in an institutional environment where reliability, security, maintainability, portability, and policy compliance are key requirements.

1.3 Problem Statement

Utrecht University already maintains comprehensive institutional research information, including publications, researcher profiles, organizational structures, and related metadata. The core challenge is not the absence of data, but the effort required to translate existing data into user-ready outputs. Common tasks such as drafting a researcher profile, compiling a narrative CV, producing an overview of departmental collaborations, or preparing policy-oriented summaries typically involve manual selection, synthesis, and interpretation. This creates friction for users who need results quickly and in a structured form that can be reused for reporting, communication, evaluation, and strategic analysis. In addition, AI assistance in a university context must be controlled and transparent. The system should provide source attribution, clearly indicate limitations, and avoid overconfident output when data is insufficient (e.g., explicit “no answer possible” behavior). Access to information must also respect authentication and authorization boundaries.

1.4 Role of Pure, Ricgraph, and OpenAlex

The system combines multiple existing sources as part of a single user-facing portal:

- Pure serves as the institutional Current Research Information System (CRIS) and provides the primary, authoritative dataset on researchers, publications, and organizational structures [2]. It forms the foundation for all generated outputs.

- Ricgraph represents research information as a graph of entities and relationships, enabling the analysis of collaborations, co-authorships, and organizational connections [10]. It supports queries focused on relational and network-based insights.
- OpenAlex is an external open-data source that provides publication meta-data and abstracts [9], used to enrich institutional data. Its abstracts are essential for the Retrieval-Augmented Generation (RAG) pipeline, enabling semantic retrieval and content-based AI responses.

Together, these systems provide institutional data (Pure), relational context (Ricgraph), and semantic enrichment (OpenAlex), allowing the Research AI to generate accurate, context-aware, and data-grounded outputs.

2 Stakeholders and Their Interests (Jonathan)

2.1 Utrecht University (ITS) - client

ITS is the commissioning body and expected long-term maintainer. Key interests include a production-ready and maintainable architecture, successful integration with existing research information systems, transparent and policy-aware AI behavior, and a portable open-source solution that can be adopted by other institutions. A common theme among these interests is the importance of well-designed architecture and clean, high-level code.

2.2 Research Staff

Important research staff (i.e. research managers, policy advisors, research support staff, and communication advisors) use research information for evaluation, strategic planning, funding preparation, and internal reporting. Their interests include efficient generation of organization-level summaries, collaboration overviews, and decision-support narratives that remain reliable and aligned with institutional policy constraints.

2.3 Researchers

Researchers are a core user group. Their interests include reducing administrative workload and quickly producing structured narratives (e.g., profile texts and CVs) grounded in verified institutional information, as well as discovering relevant publications and collaboration context. Thus, transparency and clear traceability through sources are essential to meet their expectations.

2.4 Universities (Future Adopters)

Because the system is intended to be open source and portable, other universities constitute a future stakeholder group. Their interests include clear documentation, configuration-driven deployment, no Utrecht-specific hardcoding,

and compatibility with different CRIS (Current Research Information System, a database used by institutions to manage and report on their research activities) and identity infrastructures.

3 MoSCoW Prioritization and Key Epics (Lukas/Onno)

3.1 Must Haves

- A basic and usable UI design
- Good performance and optimizations (e.g. no more than 20 seconds loading time)
- Integration with an LLM (Large Language Model, an AI system trained on large amounts of text data to understand and generate human language [11])
- Integration with Ricgraph
- Integration with Pure/OpenAlex
- Authentication via SURFconext (to protect sensitive data)
- Contain a set of pre-defined prompts for common research and policy use cases
- Find the top 20 organizations a faculty collaborates with
- Find publications, researchers, and organizations relevant to a researcher's own work (through abstract similarity)
- Generate a CV for a UU profile based on the researcher's publications
- Build a Neo4j database (a graph database that stores data as nodes and relationships rather than in traditional tables [8]) for abstracts
- RAG retrieval pipeline (Retrieval-Augmented Generation, a technique that enhances LLM responses by first retrieving relevant documents and providing them as context to the model [6])
- Search for persons/organizations

3.2 Should Haves

- Bilingual design (EN and NL, with more languages possible later)
- Mobile-responsive UI
- Filtering of results (e.g., by year)
- Display individual researcher collaborations

- LLM portability
- Show lists of collaborating researchers directly, without relying on the LLM
- General portability (usable by other universities)
- Produce a list of research outputs with overlapping topics
- Accountability section (explaining how the system arrived at its answer)
- Handling uncertainty (“no answer possible” from AI)
- Consistent AI outputs (similar prompts should produce reasonably similar outputs)

3.3 Could Haves

- UI redesign
- Clickable right pane (which enables easier relational navigation)
- User-facing AI model selection
- Improve the filtering algorithm for what gets sent to the LLM (prototype: top 10 + outliers)
- Allow users to write their own prompts
- Import media, prizes, and activities data
- Improve the search function for persons/organizations
- User-dependent prompts (one-click prompt submission based on the user’s background)
- Refine the set of standard prompts
- Visualization of collaboration networks
- Visualization of other data
- Support for showing open calls

3.4 Won’t Haves

- Extended conversations with AI
- Admin panel for user statistics

3.5 Key Epics

3.5.1 User authentication

- **Goal:** Protect UU SURF client data and ensure secure access for authorized users. Since the intended data of the client is not in the public domain, it must be protected for privacy reasons.
- **In-scope:** Account sign-in and sign-out; secure data storage.
- **Out-of-scope:** Different user-based functionality.
- **Success:** Unauthorized users cannot access the full web app and cannot access any external backend functionality.

3.5.2 Usability

- **Goal:** The software should be usable for a diverse group of professionals from the university.
- **In-scope:** At least Dutch and English UI localisation; a clear, minimal, professional interface; basic accessibility (keyboard navigation, contrast).
- **Out-of-scope:** Extensive localisation options; user-adjustable interfaces.
- **Success:** UI is available in both languages; positive feedback from the initial user group.

3.5.3 AI interaction

- **Goal:** Provide controlled AI responses via predefined prompts while reducing hallucinations (instances where the model generates plausible-sounding but factually incorrect information [5]).
- **In-scope:** UI buttons that provide predefined prompts; an input box for a single-use custom prompt.
- **Out-of-scope:** A full conversation interface with the AI; functionality to give feedback on prompts.
- **Success:** No conversation box is available to users; for manual prompts, a check verifies whether answers can be sufficiently generated using only the available RAG data.

3.5.4 Product portability & modular integrations

- **Goal:** Ensure the product can be deployed across different university environments and integrate modularly with data sources and AI providers.
- **In-scope:** Containerized deployment (Podman); API-driven design; configuration-based connections for the data aggregator (RicGraph) and AI.

- **Out-of-scope:** Extensive deployment options; user-based preference settings.
- **Success:** The service can be deployed on a different server using the provided documentation; an AI model can be switched (with privacy considerations).

3.5.5 Search functionality

- **Goal:** Users can search for an instance (e.g., a department or person) and view raw relevant information. Interacting with predefined buttons generates AI output based on that information.
- **In-scope:** A view showing raw relevant data; prompt-driven AI answer per selected button; time-range filter to narrow search results.
- **Out-of-scope:** Prompt-based entity searching; highly customized filtering (potentially for later stages).
- **Success:** An easy-to-use detailed overview is implemented and approved by external users; AI-generated responses are tested by the initial user group.

3.5.6 Documentation

- **Goal:** Provide end-user and developer documentation for usability and maintainability.
- **In-scope:** User guide; architecture documentation; argumentation for key design decisions.
- **Out-of-scope:** A feature-length film about the project.
- **Success:** Documentation covers APIs; the client finds developer documentation satisfactory; a new user can use core features within 20 minutes.

3.5.7 Phone (mobile) adaptation

- **Goal:** Ensure the web app is usable on phones with an appropriate front-end experience.
- **In-scope:** Redesigned layout for vertical screens; touch-friendly controls.
- **Out-of-scope:** Native mobile apps; offline mobile features.
- **Success:** Multiple mobile viewports are tested; all standard functionality remains accessible.

3.5.8 Quality assurance

- **Goal:** Maintain a professional quality level through systematic testing and user feedback.
- **In-scope:** Test plan; CI pipeline (Continuous Integration, a practice where code changes are automatically built and tested upon each commit [1]) with automated tests; usability testing by external users.
- **Out-of-scope:** Third-party quality testing.
- **Success:** User-reported issues are implemented or fixed.

4 MVP Description (Lukas)

4.1 Overview

The MVP (Minimum Viable Product) represents the earliest version of the system that functions as a coherent product. It is not the final delivery but rather it is the foundation that demonstrates the core concept is viable end-to-end. All development after this builds on top of the MVP by adding features, polishing, and adding robustness as defined in the MoSCoW prioritization (see Section 3 regarding "could have").

The MVP exists to answer one question: can we combine institutional and research data from multiple groups, universities, and sources to generate meaningful, AI-assisted textual outputs through a single web interface? If the answer is yes, the remaining sprints focus on refinement. If a fundamental problem exists, the MVP surfaces it early enough to change course.

4.2 What does the MVP Deliver?

The MVP is a web application that allows a user to look up a researcher or organization, view their associated publications, and generate AI-written summaries or narratives based on that publication data. The system pulls information from Pure, OpenAlex, and Ricgraph, stores abstracts in a Neo4j database when the system is initially deployed, and uses a RAG pipeline to feed relevant context to an LLM when generating output.

In concrete terms, the MVP supports three primary use cases:

- Generating a narrative CV for a researcher based on their publication history.
- Identifying the top collaborating organizations for a given faculty or department.
- Finding publications relevant to a researcher's own work through abstract similarity.

These use cases are driven by a set of predefined prompts. The user selects an entity, picks a prompt, and receives a generated output grounded in the retrieved data. The interface is minimal and functional, providing search, entity display, prompt selection, and output presentation without visual refinement or mobile support. While the product may inherently support both, this has not been a deliberate design consideration, as it falls under the could-have requirements.

4.3 Technical Foundation

The MVP integrates several components into a single working pipeline:

- **Data layer:** Publication metadata and abstracts are harvested from Pure and enriched with contextual information (e.g., fields of science) from OpenAlex. Ricgraph provides relationship data for collaboration-based queries. Abstracts are stored in a Neo4j database in both plaintext and embedded form (as vector embeddings, which are numerical representations of text that capture semantic meaning and enable mathematical similarity comparisons between documents [7]) to support efficient retrieval. This database is not exposed to the outside world, it is only accessible from internal systems, and users cannot query it directly.
- **RAG pipeline:** When a prompt is executed, the system retrieves the most relevant abstracts and metadata from Neo4j using vector similarity search (comparing the mathematical distance between embeddings to find semantically related content) and includes them as context in the request to the LLM. This grounds the output in actual institutional data rather than the model's general knowledge. Within the MVP, this pipeline uses a straightforward retrieval approach. Later iterations could incorporate more sophisticated techniques such as HyDE (Hypothetical Document Embeddings, a technique where the LLM first generates a hypothetical answer to the query, which is then embedded and used for retrieval instead of the original question, bridging the semantic gap between questions and answers [4]) or other advanced retrieval strategies to improve the relevance and quality of results.
- **LLM integration:** The system connects to a single configured language model. The architecture is designed to allow swapping the model in later iterations, but the MVP does not expose model selection to the user.
- **Web interface:** A basic frontend provides search, entity selection, prompt selection, and output display. It is built for desktop use and prioritizes functionality over aesthetics.

4.4 What the MVP Does Not Include

The MVP is deliberately minimal. The following are recognized as valuable but are not part of the initial version:

- Authentication or authorization; the MVP does not enforce user login or access enforce any control to the data once the frontend is accessed.
- Mobile-responsive design.
- Accountability features such as detailed source attribution sections or explicit uncertainty handling.
- LLM portability or organizational portability.
- Custom user-written prompts, advanced filtering, or collaboration visualizations.

These features are addressed in later sprints as part of the Should Have and Could Have priorities.

4.5 Success Criteria

The MVP is considered successful when the system can demonstrate the following as a complete, working chain:

1. A researcher or organization can be found and selected through the search interface.
2. Relevant publications and metadata are retrieved and displayed from the connected data sources.
3. A predefined prompt produces an AI-generated output (e.g, a narrative CV) that is visibly based on the retrieved publication data.
4. The RAG pipeline retrieves relevant abstracts from Neo4j and provides them as context to the LLM.

If these criteria are met, the system has proven its core concept and is ready to be extended with the features outlined in the remaining sprints.

5 Scrum implementation (Ivan)

To properly implement the scrum workflow we searched for several moments during the week where the whole group could come together and collaborate. This became Monday from 13:00 and Wednesday from 11:00. Various people have lectures and other obligations on those days, but a large part of our team is present all day on these days. Tuesdays are viewed as "extra days", where there will likely be people in the office, but others might work from home or have other things to do. The rest of the week, we are all quite busy with other things. We've found out that this schedule, which has three consecutive days that are almost fully dedicated to this project, works really well. Another benefit of this schedule is that, because most of us are (physically) present, collaboration and communication are stimulated.

- **Daily standups/scrums:** We have daily standups on the first three days of the week, because those are our working days. We planned these such that on Mondays and Wednesday everyone can be present, and Tuesday only the people that work that day have to be present. Because we only begin working together on Monday at 13:00 and our daily standup is at 10:00, it is also possible to join this standup online.
- **Sprint Review:** Sprints are two weeks long, and we end/start a sprint on Wednesdays in odd weeks. This Wednesday is full of meetings; at 10:30 we begin with a team meeting to discuss what we want to present to the client and what we are roughly planning to do next sprint. Then at 11:15 we have the sprint review together with the client, where we discuss what we have done in the past sprint, and what we are planning to do in the coming two weeks. The client can have feedback during this meeting, and together we can change priorities, discuss new feature requests, and decide what we will be working on in the next sprint.
- **Sprint Retrospective:** After the sprint review is over, we have the sprint retrospective. Here we look back at what we could have done differently, how we can change our processes, and we also discuss what went well such that we can replicate that in the next sprint. Then we take a small break before we begin with the sprint planning of the next two weeks.
- **Sprint Planning:** During the previous sprint, we already sat with a group of 3 (one backend, one frontend, and one management) to do some refinement: we take the tasks from the backlog (the prioritized list of all remaining work items for the project) and explain them in as much detail as possible. By doing this, our sprint planning becomes a lot faster and more efficient as we can easily split features into smaller tasks. Next we give story points (a relative unit of estimation used in Scrum to express the overall effort required for a task) to each individual task, and divide them among the development team to determine who does what.
- **Client Meetings:** Once a week, on Mondays, the Product Owner has a meeting with one of the clients. This has been implemented to ensure a fast and consistent way to communicate and discuss technical questions with the client. This eliminates the need to potentially wait for 2 weeks until the next sprint review, where we see the client again. These meetings are not for changing priorities, discussing what we will be doing in this sprint, or to show our product. It is only used for technical discussions.
- **Management meetings:** Once a week we've scheduled a management meeting, including the Scrum Master, Product Owner and Chairman. In the even weeks this also includes our supervisor Ioana. In these meetings we discuss how things are progressing, whether people are still working well together, and we make various big-picture overviews for the project that we can discuss with the team, such as a long-term planning of the project.

6 Risks (Timon)

This section outlines the main risks identified for the project, grouped by category. Each risk is assigned a likelihood and impact level (see Table 1), and includes a mitigation strategy.

6.1 Stakeholder Risks

- **S1 – Changing requirements:** Stakeholders may change expectations during the project.

Mitigation: Frequent communication, sprint reviews, and iterative feedback cycles.

- **S2 – Unclear expectations:** Stakeholder requirements may be insufficiently defined.

Mitigation: Prepare targeted questions and use demos to clarify expectations early.

6.2 Communication Risks

- **C1 – Limited client communication:** Interaction with the client may be too infrequent.

Mitigation: Weekly meetings with the client in addition to sprint reviews.

- **C2 – Knowledge gap with client:** The client may communicate at a higher technical level.

Mitigation: Ensure team members understand all domains and ask clarifying questions.

- **C3 – Internal miscommunication:** Team members may not properly communicate progress.

Mitigation: Establish clear communication rules and use daily stand-ups.

- **C4 – Conflict within the team:** Disagreements may negatively affect collaboration.

Mitigation: Maintain agreed communication norms and resolve conflicts constructively.

6.3 Data Risks

- **D1 – Privacy and legal constraints:** Sensitive data may not be publicly accessible.

Mitigation: Implement secure storage and authentication mechanisms.

6.4 Knowledge Risks

- **K1 – Incorrect domain understanding:** The team may misunderstand domain concepts.

Mitigation: Ensure at least two team members understand each domain and validate with the client.

6.5 Technical Risks

- **T1 – System failure:** Server or infrastructure failures may occur.

Mitigation: Use reliable infrastructure and backup strategies.

- **T2 – Authentication issues:** Problems with SURF authentication may delay progress.

Mitigation: Plan sufficient buffer time and coordinate with SURF support.

- **T3 – Rework due to technical errors:** Architectural mistakes or bugs may require code rewrites.

Mitigation: Enforce code reviews and testing before deployment.

Likelihood / Consequence	Negligible	Minor	Moderate	Major	Substantial
Almost certain				D1	C3
Likely		C1			T3
Possible		C2		S2	S1
Unlikely					C4 & K1
Rare					T1 & T2

Table 1: Risk Matrix

7 Global Planning (Onno)

This section describes what milestones the team will foresee in the future, and contains the most important moments from the agenda.

- **Sprint 1: Functional walking skeleton (11 Feb – 25 Feb)**

This first week will be dedicated to setting up the minimal requirements and dependencies of the project. A functional walking skeleton is a minimal end-to-end implementation that connects all major architectural components with the simplest possible functionality, proving that the integration works before features are built on top of it. This is an important milestone, since it will represent the backbone of the project. It is also important for transparency reasons, as this is the best time the architecture can still be changed with relative ease. The last Friday (27 Feb) marks the date where the team will deliver a project plan to the customer and the university, giving both the team and the client a clearer image of our expectations of this software project.

- **Sprint 2: Concise architecture (25 Feb – 11 Mar)**

As a follow up to the functional walking skeleton, the team will deliver a document containing the architecture of the software and design decisions. Each decision will be elaborated on, such that the client or future developers have a clear picture of why we made certain technical decisions. The official document deadline is Mar 13.

- **Sprint 3: Setup quality assurance (11 Mar – 25 Mar)**

The team will devise a plan containing our methods of quality assurance. This document is mainly focused on the technical quality of the code base and includes things like coding standards, unit tests, integration tests and our methods of implementation (CI). Our findings will be presented in a session, with the purpose of discussing the mentioned QA points. This presentation lasts for roughly 15 minutes, and is intended for the team and the teacher responsible for testing.

At this point the full RAG pipeline is implemented, allowing for searches on publicly available data. This will finalize the MVP.

This sprint will include a quality inspection by a student-assistant, of which further details are unknown.

- **Sprint 4: Testing plan (25 Mar – 8 Apr)**

On Mar 27 the team will officially hand over the plan containing all the steps we need to take to assure the quality of the software to the client and the university.

SURF based authentication will be fully implemented at the end of this period. At this point the software is able to make use of the data from Utrecht University.

- **Sprint 5: Core functionality completion (8 Apr – 22 Apr)**

Focus: Completing all Must Have functionality and stabilizing the MVP.

Key milestone: Halfway product delivery to the client.

At the end of this sprint:

- All Must Have requirements are implemented (authentication, search, predefined prompts, RAG pipeline).
- Integration with Pure, Ricgraph, and OpenAlex is fully functional.
- Core use cases (CV generation, collaboration analysis, similarity search) work end-to-end.
- A functional version of the system is delivered to the client, including source code and initial documentation.

- **Sprint 6: System refinement and Should Haves (22 Apr – 6 May)(Under discussion with client for more features)**

Focus: Improving usability and extending functionality.

Key milestone: Scrum presentation and intermediate system evaluation.

At the end of this sprint:

- Key Should Have features are implemented (filtering, collaboration views, improved UI).
- AI outputs are more consistent and better aligned with prompts.
- Feedback from the halfway delivery is incorporated into the system.

- **Sprint 7: Validation and user testing (6 May – 20 May)(Under discussion with client for more features)**

Focus: Testing the system with users and improving reliability.

Key milestone: Quality inspection and validated system behavior.

At the end of this sprint:

- User testing sessions are conducted with research staff.
- Identified usability and correctness issues are resolved.
- Quality assurance (tests, CI, stability) is strengthened.

- **Sprint 8: Final feature completion and polishing (20 May – 3 Jun)(Under discussion with client for more features)**

Focus: Completing remaining features and preparing for delivery.

Key milestone: Feature-complete system.

At the end of this sprint:

- Remaining high-priority Should Have features are completed.
- UI and performance improvements are implemented.
- The system is feature-complete and ready for final validation.

- **Sprint 9: Documentation and final validation (3 Jun – 17 Jun)**

Focus: Preparing the system for handover.

Key milestone: Draft final report submission (June 5).

At the end of this sprint:

- Final documentation (user guide, architecture, deployment) is completed.
- The system is validated against success criteria.
- A stable release version is prepared.
- **Sprint 10: Final delivery (17 Jun – 24 Jun)**
Focus: Delivery and presentation.
Key milestones: Final report (June 22), final presentation (June 23), product delivery (June 24).
 At the end of this sprint:
 - Final report is submitted.
 - Final presentation and demonstration are completed.
 - The final product is delivered, including software, documentation, and release notes.

8 Technical and Domain Knowledge (Anouar)

8.1 Domain knowledge

For this project we need to work with multiple different systems. To make sure the project goes smoothly we need to have a good understanding of these systems. These systems are:

- **RicGraph:** We need to understand how RicGraph works and how it models research information as a knowledge graph (a structured representation of real-world entities and the relationships between them, stored as nodes and edges in a graph database). We must understand which nodes and edges exist, what relationships mean (e.g., collaboration, affiliation, co-authorship), and how to query the graph efficiently and responsibly.
- **Pure:** We need to know how to retrieve abstracts reliably from Pure. These abstracts will be used for the RAG responses.
- **OpenAlex:** We need knowledge of OpenAlex as an external open-data source used for enrichment.
- **SURF:** We need to understand SURF for authentication and authorization. We must know how login is performed, and how authorization constraints affect the full data flow.

8.2 Technical knowledge

Technically, we need knowledge of the full stack and how the services work together. Below is the full tech stack that we will be using for this project:

- The frontend is built with React and TypeScript, with i18next (an internationalization framework that enables switching between language translations at runtime) for bilingual UI (EN/NL) and Axios (an HTTP client library used to make API requests from the browser) for API calls.
- The backend is Python with FastAPI (a modern, high-performance web framework for building APIs in Python) serving request/response schemas, embeddings, autocomplete, and graph queries.
- For graph functionality we use Neo4j with Cypher (Neo4j's declarative graph query language, used to express pattern-based queries over nodes and relationships [3]) because RicGraph is backed by Neo4j.
- Everything will be containerized (packaged into isolated, reproducible units called containers that bundle the application with all its dependencies) with Podman for development, served behind Caddy as reverse proxy (a server that sits between clients and backend services, forwarding requests and handling concerns such as TLS termination and load balancing), and deployed on Linux with systemd Quadlet (a systemd integration that allows managing containers as native Linux services).

At the start of the project (Sprint 0), we will focus on building an understanding of the systems and technologies we will use. We will mainly do this by reading into the systems. Team members who already have experience with specific technical components will explain them to the rest of the group during this sprint. This ensures we align early on the architecture and the right implementation approach. Throughout the project, when we encounter issues where we lack knowledge, we will account for that in the sprint planning, for example by adding time for learning and assigning additional story points to those tasks.

9 AI Statement (Jonathan)

For this project, our team follows Level 4 of the Utrecht University AI Index. Beyond these formal requirements, we do not apply additional strict restrictions on AI use. However, we find responsible and transparent use of AI important. When a team member uses AI tools, they remain fully responsible for the content that is produced. This means that each team member must:

- Understand what they submit as part of the project (e.g., text, code, analyses);
- Be able to explain and justify the choices made in that work;
- Review AI-generated output critically and correct it where needed;
- Ensure that the final deliverables reflect the team member's own understanding and accountability.

References

- [1] Paul M. Duvall, Steve Matyas, and Andrew Glover. *Continuous Integration: Improving Software Quality and Reducing Risk*. Addison-Wesley Professional, 2007.
- [2] Elsevier. *Pure Research Information System*. 2024. URL: <https://www.elsevier.com/products/pure>.
- [3] Nadime Francis et al. “Cypher: An Evolving Query Language for Property Graphs”. In: *Proceedings of the 2018 International Conference on Management of Data* (2018), pp. 1433–1445.
- [4] Luyu Gao et al. “Precise Zero-Shot Dense Retrieval without Relevance Labels”. In: *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics*. 2023. URL: <https://arxiv.org/abs/2212.10496>.
- [5] Lei Huang et al. “A Survey on Hallucination in Large Language Models: Principles, Taxonomy, Challenges, and Open Questions”. In: *arXiv preprint arXiv:2311.05232* (2023).
- [6] Patrick Lewis et al. “Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks”. In: *Advances in Neural Information Processing Systems* 33 (2020), pp. 9459–9474.
- [7] Tomas Mikolov et al. “Efficient Estimation of Word Representations in Vector Space”. In: *arXiv preprint arXiv:1301.3781* (2013).
- [8] Neo4j, Inc. *Neo4j Vector Search*. Accessed: 2025. 2024. URL: <https://neo4j.com/docs/cypher-manual/current/indexes/semantic-indexes/vector-indexes/>.
- [9] OurResearch. *OpenAlex: Open Catalog of Scholarly Papers*. 2024. URL: <https://openalex.org>.
- [10] Utrecht University. *Ricgraph: Research in Context Graph*. 2024. URL: <https://github.com/UtrechtUniversity/ricgraph>.
- [11] Wayne Xin Zhao et al. “A Survey of Large Language Models”. In: *arXiv preprint arXiv:2303.18223* (2023).

10.3. Architecture Plan

Architecture plan — Research AI

Triple One Intelligence:

Jelmer Andre de la Porte 6578772

Ivan Balkenende 1196340

Jonathan Bosker 2046636

Anouar Boubkari 1747479

Lukas van Dee 1048708

Thijmen Ligter 4687582

Sybren van der Meijden 2077523

Onno Meppelink 0989711

Timon Zoutendijk 4611659

Friday the 13 of March 2026

1 Introduction

This project aims to develop an open-source portable web application that provides AI-assisted access to institutional research information. Its primary goal is to simplify and accelerate the exploration of research data for Utrecht University staff, researchers, and policy advisors by generating structured, reliable, and actionable outputs from existing sources. The system integrates data from internal tools such as *Pure* and *Ricgraph*, as well as external sources such as *OpenAlex*, combining publications, researcher profiles, organizational relationships, and collaboration information.

This architecture plan outlines key technical decisions, system components, and integration strategies that enable the Research AI Portal to deliver a reliable, scalable, and user-centric solution. It focuses on the structure of the system, data flow, and interaction between components, providing a foundation for both development and future enhancements.

2 Overview of architecture

The system uses a modular, container-based architecture consisting of five main components. When deployed on a single machine, services communicate directly. When distributed across multiple servers, the connections between services use SSH tunnels with port forwarding so that no service needs to expose ports to the outside. Because each service only connects to a local port, switching between a single-server and a multi-server deployment does not require changes to the application code.

- **Web frontend container:** A Caddy container that serves the compiled React frontend as static files and acts as a reverse proxy for backend requests.
- **Backend API:** A REST API built with Python 3.12 and FastAPI. It handles requests from the frontend, queries the graph database, performs vector similarity search on stored embeddings, and coordinates communication with the AI service.
- **Neo4j Database:** A Neo4j container that stores the graph data used by the application. In addition to the standard Ricgraph node structure, nodes of type DOI are extended with vector embedding and plaintext fields derived from publication abstracts, enabling semantic similarity search directly within the graph.
- **Ricgraph Service:** A separate container running Ricgraph, a tool for harvesting and linking research information from multiple sources. It populates and maintains the Neo4j graph with researcher profiles, publications, organizational relationships, and other research metadata. The system builds on Ricgraph as a client requirement.

- **AI Service:** A locally hosted AI service deployed using the standard Ollama container.

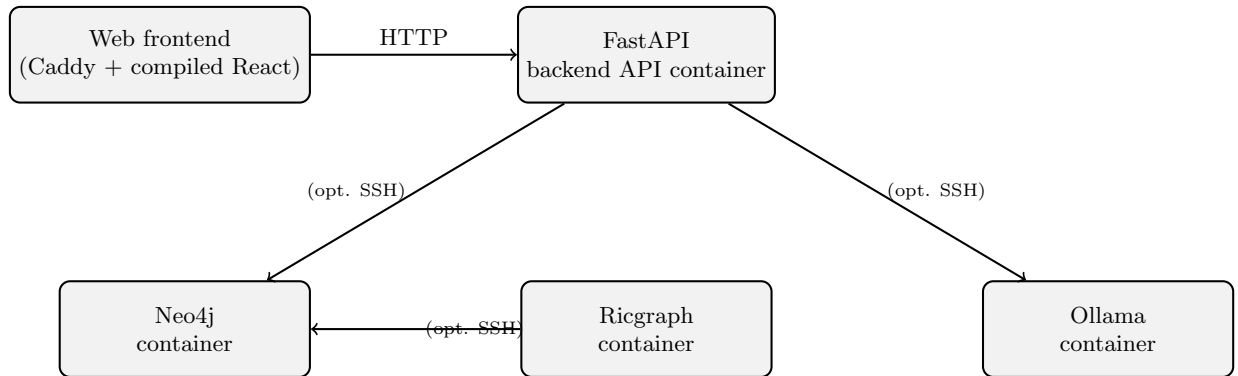


Figure 1: Container-level overview of the application. When services run on the same server they communicate directly. When distributed across multiple servers, the connections use SSH tunnels with port forwarding. The backend queries Neo4j for both graph traversals and vector similarity search. Ricgraph operates independently to harvest and maintain the graph data in Neo4j and has no direct communication with the backend at runtime.

3 Substantiated choices

- **React (Frontend):** React was a client requirement and is used for the web frontend. This is a reasonable choice, since React is widely used and has a mature ecosystem. If the framework choice had not been fixed in advance, Svelte could also have been considered as an alternative because it appears simpler to work with in some cases.
- **Python and FastAPI (Backend):** Python was chosen for the backend mainly for practical reasons. Although a language such as Go could be preferable in some respects, the team's existing experience made Python the more realistic choice within the time and scope of the project. Python also has the libraries needed for easy database queries, embedding generation, and LLM integration. FastAPI is used to implement the HTTP API through which the frontend triggers retrieval and generation.
- **Neo4j and Ricgraph (Data layer):** The use of Ricgraph is a client requirement, as the client developed Ricgraph and the project is intended to build on top of it. Ricgraph is built around Neo4j, which determines the choice of database. Neo4j is also well-suited for this use case independently: it is the most mature and widely adopted graph database currently available, and its native support for vector indexes makes it possible to

store embeddings directly on graph nodes. In this setup, Ricgraph and the application share the same Neo4j instance, which keeps the deployment simpler and avoids the overhead of running a separate database. Ricgraph runs in its own container, as it was designed and packaged to be deployed that way.

- **Ollama (Local AI):** Running the language model locally was a client requirement. The system uses the standard Ollama container so that model execution is handled by a separate service. This keeps the backend decoupled from the LLM service and makes it easier to replace the local setup later with an external API without major changes to the rest of the application.
- **Caddy (Web serving and reverse proxy):** Caddy is used to serve the compiled frontend as static files from within the web frontend container. It also acts as a reverse proxy, forwarding API requests from the frontend to the backend. Caddy was chosen over alternatives such as Nginx because it can automatically obtain and renew TLS certificates through Let's Encrypt without requiring an additional service or separate certificate management setup. Due to client requirements, the application is currently deployed locally on a SURF Research Cloud VM, so the TLS functionality is not actively used in the current deployment. However, it remains in place to simplify a possible future transition to a publicly accessible deployment should the legal situation around the aggregated data be resolved.
- **Podman and Systemd Quadlet (Deployment):** Podman and Systemd Quadlet were chosen instead of Docker for container deployment. Docker relies on a central daemon, which effectively reimplements functionality that is already provided by systemd. Since the host system already uses systemd as its init system, as do many Linux distributions, it made more sense to use Quadlet to manage container processes directly as system services.
- **SSH tunneling and port forwarding (Inter-service communication):** When services are deployed across multiple servers, communication between them uses SSH tunnels with port forwarding. Each service connects to a local forwarded port rather than directly to a remote host, so no service needs to expose any port to the outside network. When all services run on the same server, they communicate directly and SSH is not involved. Because the application always connects to a local port regardless of the deployment setup, moving a service to a different server only requires setting up an SSH tunnel for that connection. This makes it possible to scale the system to multiple machines without changing the application itself. Development machines also use SSH port forwarding to connect to services running on remote servers, which keeps the access method consistent across environments.

4 Non-functional requirements

Non-functional requirements describe how well a system performs its tasks and address aspects such as performance, reliability, security, usability, maintainability, and portability. According to the ISO/IEC 25010 software quality model, these characteristics are essential for evaluating the overall quality of a software system. The following subsections detail the specific quality attributes that apply to this project.

4.1 Performance

Users expect that search queries and AI-generated responses are delivered within a reasonable time. Long response times negatively affect the usability and overall user experience of the application.

The system must therefore be designed to retrieve and process relevant research information efficiently. When a user executes a prompt, the application retrieves relevant publications and metadata from the graph database and provides this information as context for the language model. An acceptable upper bound for total response time has been set at 20 seconds to ensure a smooth user experience.

To support these performance requirements, the system uses vector embeddings stored directly on Neo4j nodes. Publication abstracts are converted into vector representations that capture semantic meaning. When a user submits a query, the backend performs a vector similarity search on the DOI nodes in the graph to identify the most relevant documents. This avoids full-text searches across the entire dataset and improves the efficiency of the retrieval process within the Retrieval-Augmented Generation (RAG) pipeline.

4.2 Reliability

Reliability is a critical requirement of the system, as users must be able to trust the generated results. The application must operate in a stable manner and produce consistent output based on the available data.

Since the system relies on multiple external data sources, it must be able to handle missing or temporarily unavailable data gracefully. If an external data source cannot be accessed or if insufficient information is available to generate an answer, the system should clearly communicate this to the user.

Furthermore, the system must reduce the risk of generating incorrect or fabricated information. By using a Retrieval-Augmented Generation (RAG) architecture, the language model is guided by retrieved documents from the database rather than relying solely on its pre-trained knowledge. This approach reduces the likelihood of hallucinations and improves the reliability of generated outputs.

4.3 Security and Privacy

Because the system processes research data originating from institutional databases, security and privacy are important non-functional requirements.

The data served by the application is aggregated from multiple individually open data sources such as Pure, OpenAlex, and institutional repositories. While each source may be publicly accessible on its own, the combination of these datasets into a single searchable system raises unresolved legal questions. Several universities have taken the position that such aggregation requires explicit authorization, and this issue has not yet been fully clarified for the data used in this project. As a result, the system is developed under the assumption that the aggregated data should not be publicly exposed.

To address this, the application is deployed on a SURF Research Cloud (SRC) virtual machine. Access to the application is restricted to authenticated users who connect to the VM through the SURF Research Cloud portal, which provides built-in authentication via SURFConext. The application itself is only reachable on localhost within the VM, so no endpoints are exposed to the public internet.

All communication between services uses SSH tunnels with port forwarding when they are distributed across multiple servers. In the current single-VM deployment, services communicate directly and no ports are exposed beyond the VM itself. The Neo4j database is only reachable from the backend and the Ricgraph container, and the Ollama service is only reachable from the backend.

This deployment model provides strong isolation but comes at the cost of usability, since users must go through the SURF Research Cloud portal, log in, and navigate to the VM before they can use the application. The architecture has been designed so that if the legal situation is resolved in the future, the system can be transitioned to a publicly accessible deployment with OAuth 2.0 for authentication and TLS handled by the existing Caddy configuration. Because each service only connects to a local port, the system can also be distributed across multiple servers by adding SSH tunnels without changing the application code. The current security constraints are therefore a deployment-level restriction rather than an architectural limitation.

4.4 Usability

The system is designed for researchers, research administrators, and policy staff who need quick access to research information. Therefore, the application must provide a clear and intuitive user interface.

Users should be able to easily search for a researcher or organization and select a prompt to generate relevant information. The overall workflow of the system should remain straightforward and require minimal steps to obtain results.

The generated output should also be presented in a clear and structured manner so users can quickly understand and interpret the information. Although the initial version of the system prioritizes functionality over advanced

interface design, usability remains an important quality requirement.

4.5 Maintainability

The system must be designed in a way that supports future maintenance and development. AI technologies and research data infrastructures evolve rapidly, which means that the software should be adaptable to new technologies, models, or data sources.

To achieve this, the system is built using a modular architecture. Core components such as the database, retrieval pipeline, AI integration, and web interface are developed as separate modules. This separation allows individual components to be modified or replaced without affecting the entire system.

Additionally, the codebase should be structured in a clear and consistent manner. The use of meaningful naming conventions, consistent coding standards, and adequate documentation helps ensure that other developers can understand and extend the system more easily.

4.6 Portability

Another important requirement of the system is portability. The goal of the project is not only to create a solution for a single university but also to design a system that can potentially be adopted by other universities or research institutions.

To support this goal, the software is designed in a generic and flexible manner. The application does not depend on infrastructure or internal systems that are unique to a single institution. Integrations with data sources such as Pure, Ricgraph, and OpenAlex are configurable rather than hardcoded in the system.

This approach allows other universities to connect their own research data sources without requiring major changes to the application's core codebase.

Furthermore, the system is able to operate in different IT environments, including local servers or cloud-based infrastructures. By using standardized technologies and a clearly defined architecture, the system becomes easier to deploy in different institutional contexts.

Comprehensive documentation also plays an essential role in ensuring portability. Installation instructions, configuration settings, and architectural decisions are clearly documented so that other institutions can independently deploy and adapt the system to their own research environments.

5 Appendix

5.1 Coding Standards

5.1.1 Code Comments

- Every function must include a description explaining its purpose and behavior.

- Every file must contain a brief description explaining its purpose and the functionality it provides.

5.1.2 Test Files

Test files are located in `api/tests/` and follow the naming pattern `test_<module_name>.py`. Tests are organized into classes, with each class grouping tests for a specific unit.

5.1.3 Naming Conventions for Functions, Classes, and Types

Python

- Functions use **snake_case** and describe the action, preferably as *verb + noun*. Internal helper functions start with a leading underscore.
- Classes use **PascalCase** and are singular, representing the entity modeled.
- Custom exceptions inherit from a suitable standard exception and always end with **Error**. The base class choice reflects the type of error (e.g., **RuntimeError** for unexpected runtime issues, **ValueError** for invalid input).

TypeScript & React

- Functions use **camelCase** and describe the action performed.
- Event handlers start with **handle** followed by the element or action they respond to.
- React components use **PascalCase** and describe what they visually or functionally represent.
- Types describe the data structure, are singular, and as concrete as possible.

5.2 Definition of Done

A feature is considered complete when the following criteria are met:

- The new feature works and is presentable to the client.
- The new feature has tests written for it (if it is backend functionality).
- The tests for the new feature pass successfully (if it is backend functionality).
- The new feature is code reviewed by at least two other developers (request a review in the Discord **front-end** or **backend** channel).

6 References

ISO/IEC. (2011). *ISO/IEC 25010: Systems and software engineering — Systems and software Quality Requirements and Evaluation (SQuaRE) — System and software quality models*. International Organization for Standardization.

10.4. Test Plan

Test plan — Research AI

Triple One Intelligence:

Jelmer Andre de la Porte 6578772

Ivan Balkenende 1196340

Jonathan Bosker 2046636

Anouar Boubkari 1747479

Lukas van Dee 1048708

Thijmen Ligter 4687582

Sybren van der Meijden 2077523

Onno Meppelink 0989711

Timon Zoutendijk 4611659

Sunday the 21 of June 2026

1 Architecture (Jelmer)

The system is a client-server web application that provides AI-assisted access to institutional research information. The application consists of five containers: Caddy serves the React frontend and proxies API calls to the FastAPI backend, which queries Neo4j and calls Ollama. Ricgraph writes to Neo4j independently and is not involved in handling user queries. When a user submits a query, the backend embeds it, retrieves matching publications from Neo4j through a vector similarity search, and passes them as context to Ollama to generate a response.

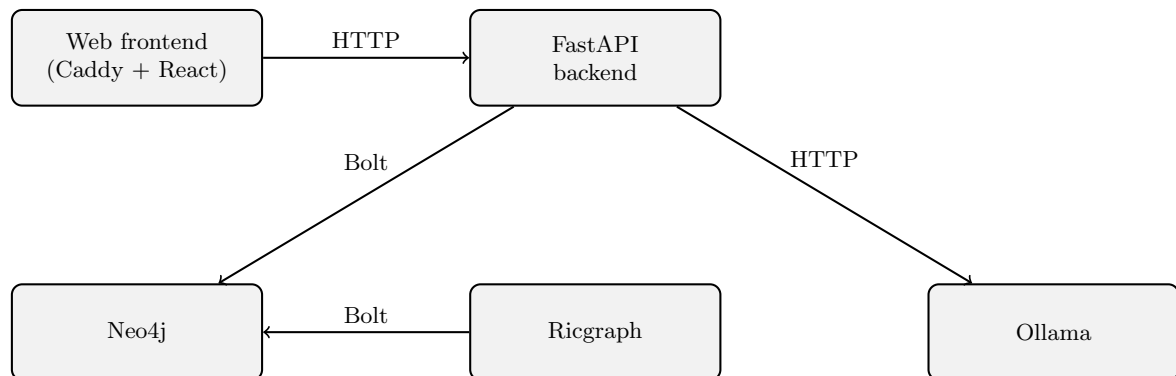


Figure 1: Container-level overview.

2 Quality aspects (Timon)

According to the ISO 25010 quality aspects, there are eight relevant aspects for our software system. The following subsections explain these aspects and the relevancy to our project.

2.1 Functional suitability

Functional suitability is directly applicable to this project, as the client has specified a set of features that must be incorporated into the website. These requirements serve as a goal, and it is the responsibility of the development team to ensure that all specified functionality is correctly implemented within the scope of this project.

2.2 Performance efficiency

Performance efficiency is a particularly relevant quality aspect in the context of this project, given the involvement of a Large Language Model (LLM). Specifically, the time between the submission of a user prompt and the display of the

LLM's reply in the middle interface column is very important to the overall user experience. Having to wait for an excessive amount of time in this interaction is unacceptable, as users expect timely feedback from the system. Because of this, a maximum response time of 20 seconds was established with the client.

2.3 Compatibility

The compatibility aspect is of high importance in this project. The client has communicated that there are long-term plans to deploy this software across universities worldwide. Therefore, it is vital that the system is designed in such a way that both the databases and the LLM models can be easily exchanged or replaced without having to redo previous work.

2.4 Usability

Usability is another critical quality aspect of this project. Given that users can obtain the information provided by the research AI platform in various other ways, it is essential that the website offers a superior and efficient user experience in order to encourage people to use the research AI platform instead of other options. We plan on extensive user testing in order to evaluate and refine the usability of the platform.

2.5 Reliability

Reliability is a key quality concern for this project, since we are working with an LLM, it is very important that the user can rely on the LLM to retrieve useful and correct information. This does not only apply to the quality of the LLM's response, but also in a more general way, that the platform is constantly available to use.

2.6 Security

Security is also an important quality aspect of this project however, the security responsibilities have already been addressed for us by the client. It has also been made clear that the client does not want us to deal with security, as we are working with confidential data and the security itself is very difficult. Thus, the project operates on a server provided by the client and is hosted through SURF, an organization that has implemented the necessary security level. As a result, the team can rely on the security standards maintained by SURF.

The user will not be able to directly prompt the LLM. Instead, there will only be pre-defined standard prompts the user can select. Because of this, prompt

injection in the LLM is not a concern for this project.

2.7 Maintainability

Maintainability is one of, if not the highest priorities of this project. Given the development period of twenty weeks and the likelihood that the project will be continued or extended by other parties afterward, the team holds a significant responsibility to produce code that is clean, well-documented, and structured in a way that allows easy modification, reuse, analysis, and testing by future developers. We will have three opportunities to get a code review from a teaching assistant. This will help keep the code to a high standard.

2.8 Portability

Portability is closely related to the maintainability and the long-term vision of this project. Given the client's objective of making the software available to universities across the world, it is essential that the system is designed to be portable. This means that it is capable of being deployed and operated across many different environments without requiring significant rework.

3 Elements of the QA-strategy (Sybren)

3.1 Functional testing strategies

In projects like ours, having well-defined test strategies across multiple abstraction levels is vital. Our testing strategies focus on the backend API and thus tests are grouped within subdirectories to reflect their abstraction levels (e.g. unit, integration and system). Within those directories are the actual files that contain the tests, which are also divided neatly into classes and functions (with appropriate names).

3.1.1 Unit tests

The most basic tests among these are unit tests. Our unit test strategy makes sure that the smallest parts of our backend API are tested as follows: we aim to create useful unit tests for all our various utility functions, schemas/classes, endpoints, and helper scripts. These unit tests involve a lot of mocking and therefore do not require real data or server access. We also try to keep them as simple as possible while being thorough and testing edge cases.

3.1.2 Integration tests

Our integration tests focus on validating that our API correctly handles incoming requests and routes requests appropriately. This obviously means that the backend code must be running (locally) such that requests can be sent and received. Our strategy is to make sure that the most basic aspects of our API calls function properly, which means that we also require connections to things like our database and the LLM. We do not validate any of the returned data, but rather check if requests are accepted or denied when expected, whether responses include any data, and that the responses are in their proper formats.

3.1.3 System tests

The most high-level tests which we use, called system tests, have the purpose of validating the entire API pipeline. Our strategy is to make API requests to the actual production server (which naturally needs to be running for this to work) and validate that we get an appropriate response which is only possible if all the required connections are established and functioning correctly.

3.2 Other QA instruments

To ensure the quality of our project, we need more than just functional tests. Listed here are some of our most important QA instruments.

- Documentation: documentation clearly defines what the code is expected to do, and thus helps us stay on track and more easily develop tests.
- Proper pull requests: by using pull requests (which come with built-in checklists and AI code review) that can only be merged if at least two collaborators approve the code, we prevent a lot of bad code from reaching our code base.
- Team agreements: we have internally discussed and documented how we wish to work together, this also includes some agreements concerning how we write code. Things like code comments, AI usage and the importance of clear structure are among the things discussed. These things will help with ensuring proper quality throughout this project.
- Quality inspections: over the course of this project, there are multiple code quality inspections by teaching assistants, this helps us get an outside look into our code base which might catch mistakes that we overlooked.

3.3 Tooling and test automation

To write our tests we use pytest (and where necessary pytest-asyncio) alongside MagicMock. To guarantee that our workflow actually uses our various tests and tools, we've incorporated them into our deployment and development processes. We use a Makefile [1] for these workflows, and have also made sure

that our Makefile automatically runs the appropriate tests. This means that when testing either locally or on a server, the developer instantly sees whether this new code passes the tests, and if not, which tests fail.

3.4 Non-functional testing

Most of the non-functional requirements (if they are testable) are hard to quantify and are therefore hard to write useful tests for. We've solved this by having at least two persons review new code to ensure quality aspects (e.g., maintainability, security), and we also employ user testing. This means letting potential future users use the system for their various purposes, collecting their feedback, and using those issues as both checklists for future versions and immediate required changes. To make sure we receive useful feedback, we will provide pre-defined goals that user wills need to achieve which confirm whether our program is usable in regular use cases. The users will also be encouraged to test their specific use cases, so that we can come up with new features and confirm our program works in other cases too.

4 Management aspect (Timon)

4.1 Inbedding

In our agile development, we have done a few things to inbed quality assurance (QA) in our system. The first being our test automation. We have set up our testing environment in a way that automatically runs the tests when code is deployed. In this way, it is easy to check whether your changes are viable and meet the expected requirements. In order to update the test environment, we made a rule in which you have to write your own tests for every task you implement, resulting in regular small updates instead of periodic large updates.

Secondly, in our definition of done in appendix A, we stated that a pull request must be reviewed by two other people. In addition, we integrated a claude bot to automatically review pull requests, saving us a lot of time when reviewing the pull requests ourselves. When issues do appear in a pull request, the issues need to be resolved first before doing a merge to the main branch.

4.2 Timeline

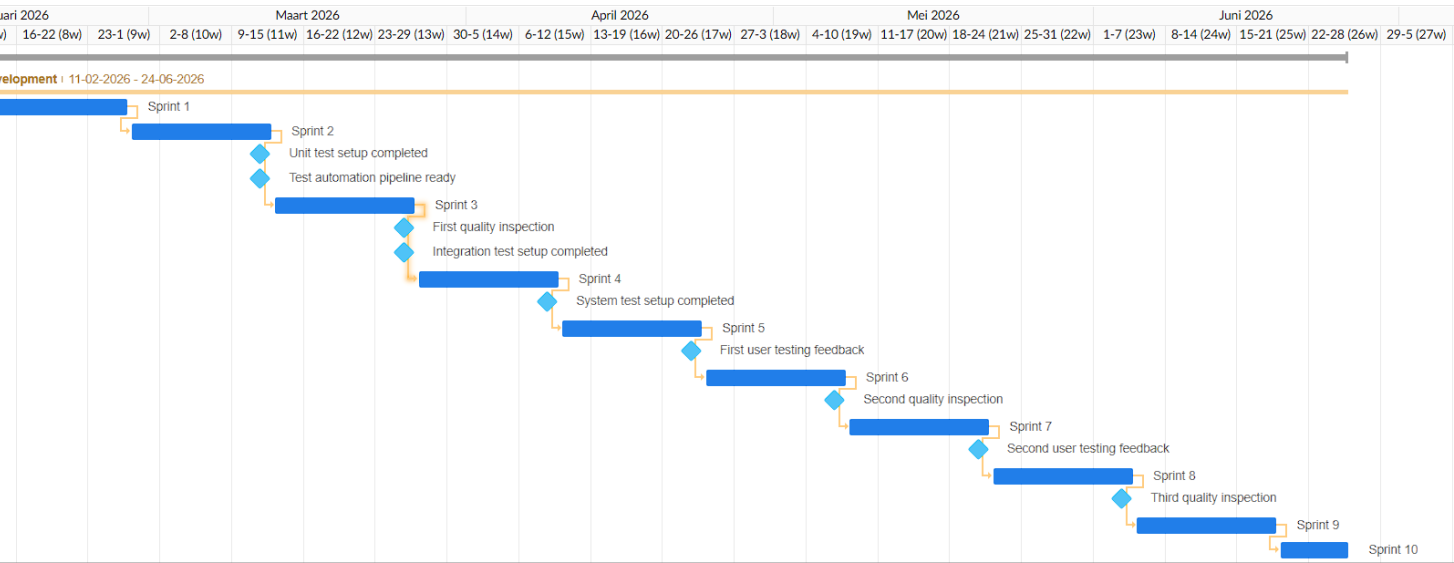


Figure 2: Gantt Chart

4.3 Resources

Since we strive to have a high quality assurance we expect to spend a significant amount of time on testing. Both when writing functional tests for our tasks and for non-functional testing. In addition, we also need to write proper documentation and review pull requests. All in all, our indication is that we spent around 30% of our time on QA.

5 Appendix

A Definition of Done

A feature is considered complete when the following criteria are met:

- The new feature works and is presentable to the client.
- The feature is documented with appropriate code comments and/or supporting documentation.
- The new feature has tests written for it (if it is backend functionality).
- The tests for the new feature pass successfully (if it is backend functionality).
- The new feature is code reviewed by at least two other developers (request a review in the Discord **front-end** or **backend** channel).

References

- [1] Free Software Foundation. *GNU make*. 2023. URL: <https://www.gnu.org/software/make/manual/make.html>.